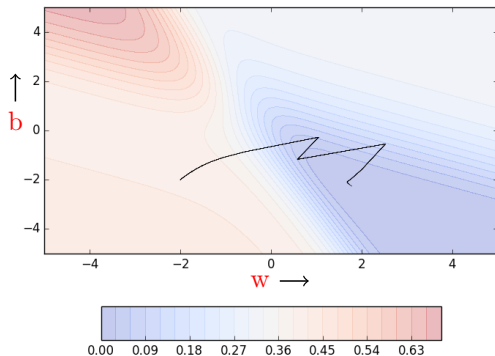
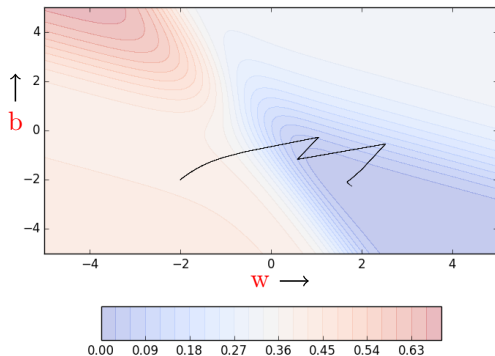


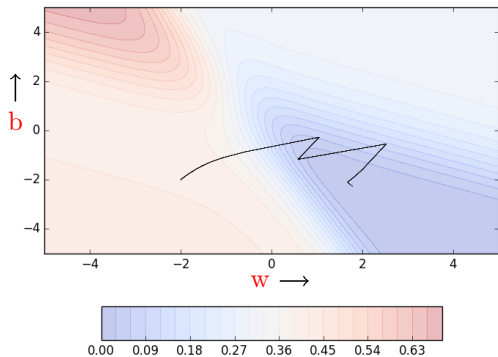
- One could argue that we could have solved the problem of navigating gentle slopes by setting the learning rate high (i.e., blow up the small gradient by multiplying it with a large η)
- Let us see what happens if we set the learning rate to 10



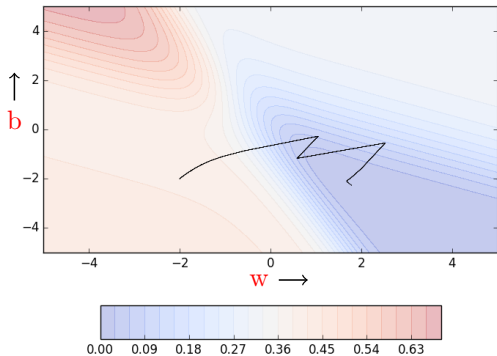
- One could argue that we could have solved the problem of navigating gentle slopes by setting the learning rate high (i.e., blow up the small gradient by multiplying it with a large η)
- Let us see what happens if we set the learning rate to 10
- On the regions which have a steep slope, the already large gradient blows up further



- One could argue that we could have solved the problem of navigating gentle slopes by setting the learning rate high (i.e., blow up the small gradient by multiplying it with a large η)
- Let us see what happens if we set the learning rate to 10
- On the regions which have a steep slope, the already large gradient blows up further
- It would be good to have a learning rate which could adjust to the gradient ...



- One could argue that we could have solved the problem of navigating gentle slopes by setting the learning rate high (i.e., blow up the small gradient by multiplying it with a large η)
- Let us see what happens if we set the learning rate to 10
- On the regions which have a steep slope, the already large gradient blows up further
- It would be good to have a learning rate which could adjust to the gradient ... we will see a few such algorithms soon



Tips for initial learning rate ?

Tips for initial learning rate ?

- Tune learning rate [Try different values on a log scale: 0.0001, 0.001, 0.01, 0.1, 1.0]

Tips for initial learning rate ?

- Tune learning rate [Try different values on a log scale: 0.0001, 0.001, 0.01, 0.1, 1.0]
- Run a few epochs with each of these and figure out a learning rate which works best

Tips for initial learning rate ?

- Tune learning rate [Try different values on a log scale: 0.0001, 0.001, 0.01, 0.1, 1.0]
- Run a few epochs with each of these and figure out a learning rate which works best
- Now do a finer search around this value [for example, if the best learning rate was 0.1 then now try some values around it: 0.05, 0.2, 0.3]

Tips for initial learning rate ?

- Tune learning rate [Try different values on a log scale: 0.0001, 0.001, 0.01, 0.1, 1.0]
- Run a few epochs with each of these and figure out a learning rate which works best
- Now do a finer search around this value [for example, if the best learning rate was 0.1 then now try some values around it: 0.05, 0.2, 0.3]
- Disclaimer: these are just heuristics ... no clear winner strategy

Tips for annealing learning rate

Tips for annealing learning rate

- **Step Decay:**

Tips for annealing learning rate

- **Step Decay:**
 - Halve the learning rate after every 5 epochs or

Tips for annealing learning rate

- **Step Decay:**

- Halve the learning rate after every 5 epochs or
- Halve the learning rate after an epoch if the validation error is more than what it was at the end of the previous epoch

Tips for annealing learning rate

- **Step Decay:**

- Halve the learning rate after every 5 epochs or
- Halve the learning rate after an epoch if the validation error is more than what it was at the end of the previous epoch

- **Exponential Decay:** $\eta = \eta_0^{-kt}$ where η_0 and k are hyperparameters and t is the step number

Tips for annealing learning rate

- **Step Decay:**

- Halve the learning rate after every 5 epochs or
- Halve the learning rate after an epoch if the validation error is more than what it was at the end of the previous epoch

- **Exponential Decay:** $\eta = \eta_0^{-kt}$ where η_0 and k are hyperparameters and t is the step number

- **1/t Decay:** $\eta = \frac{\eta_0}{1+kt}$ where η_0 and k are hyperparameters and t is the step number

Tips for momentum

- The following schedule was suggested by Sutskever *et. al.*, 2013

$$\gamma_t = \min(1 - 2^{-1 - \log_2(\lfloor t/250 \rfloor + 1)}, \gamma_{max})$$

where, γ_{max} was chosen from $\{0.999, 0.995, 0.99, 0.9, 0\}$

Module 5.8 : Line Search

Just one last thing before we move on to some other algorithms ...

- In practice, often a line search is done to find a relatively better value of η

- In practice, often a line search is done to find a relatively better value of η

```
def do_line_search_gradient_descent():  
    w, b, etas = init_w, init_b, [0.1, 0.5, 1.0, 5.0, 10.0]  
    for i in range(max_epochs):  
        dw, db = 0, 0  
        for x,y in zip(X, Y):  
            dw += grad_w(w, b, x, y)  
            db += grad_b(w, b, x, y)  
        min_error = 10000 #some large value  
        best_w, best_b = w, b  
        for eta in etas:  
            tmp_w = w - eta * dw  
            tmp_b = b - eta * db  
            if error(tmp_w, tmp_b) < min_error:  
                best_w = tmp_w  
                best_b = tmp_b  
                min_error = error(tmp_w, tmp_b)  
        w, b = best_w, best_b
```

- In practice, often a line search is done to find a relatively better value of η
- Update w using different values of η

```
def do_line_search_gradient_descent():  
    w, b, etas = init_w, init_b, [0.1, 0.5, 1.0, 5.0, 10.0]  
    for i in range(max_epochs):  
        dw, db = 0, 0  
        for x,y in zip(X, Y):  
            dw += grad_w(w, b, x, y)  
            db += grad_b(w, b, x, y)  
        min_error = 10000 #some large value  
        best_w, best_b = w, b  
        for eta in etas:  
            tmp_w = w - eta * dw  
            tmp_b = b - eta * db  
            if error(tmp_w, tmp_b) < min_error:  
                best_w = tmp_w  
                best_b = tmp_b  
                min_error = error(tmp_w, tmp_b)  
        w, b = best_w, best_b
```

- In practice, often a line search is done to find a relatively better value of η
- Update w using different values of η
- Now retain that updated value of w which gives the lowest loss

```
def do_line_search_gradient_descent():  
    w, b, etas = init_w, init_b, [0.1, 0.5, 1.0, 5.0, 10.0]  
    for i in range(max_epochs):  
        dw, db = 0, 0  
        for x,y in zip(X, Y):  
            dw += grad_w(w, b, x, y)  
            db += grad_b(w, b, x, y)  
        min_error = 10000 #some large value  
        best_w, best_b = w, b  
        for eta in etas:  
            tmp_w = w - eta * dw  
            tmp_b = b - eta * db  
            if error(tmp_w, tmp_b) < min_error:  
                best_w = tmp_w  
                best_b = tmp_b  
                min_error = error(tmp_w, tmp_b)  
        w, b = best_w, best_b
```

- In practice, often a line search is done to find a relatively better value of η
- Update w using different values of η
- Now retain that updated value of w which gives the lowest loss
- Essentially at each step we are trying to use the best η value from the available choices

```
def do_line_search_gradient_descent():  
    w, b, etas = init_w, init_b, [0.1, 0.5, 1.0, 5.0, 10.0]  
    for i in range(max_epochs):  
        dw, db = 0, 0  
        for x,y in zip(X, Y):  
            dw += grad_w(w, b, x, y)  
            db += grad_b(w, b, x, y)  
        min_error = 10000 #some large value  
        best_w, best_b = w, b  
        for eta in etas:  
            tmp_w = w - eta * dw  
            tmp_b = b - eta * db  
            if error(tmp_w, tmp_b) < min_error:  
                best_w = tmp_w  
                best_b = tmp_b  
                min_error = error(tmp_w, tmp_b)  
        w, b = best_w, best_b
```

- In practice, often a line search is done to find a relatively better value of η
- Update w using different values of η
- Now retain that updated value of w which gives the lowest loss
- Essentially at each step we are trying to use the best η value from the available choices
- What's the flipside?

```
def do_line_search_gradient_descent():
    w, b, etas = init_w, init_b, [0.1, 0.5, 1.0, 5.0, 10.0]
    for i in range(max_epochs):
        dw, db = 0, 0
        for x,y in zip(X, Y):
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)
        min_error = 10000 #some large value
        best_w, best_b = w, b
        for eta in etas:
            tmp_w = w - eta * dw
            tmp_b = b - eta * db
            if error(tmp_w, tmp_b) < min_error:
                best_w = tmp_w
                best_b = tmp_b
                min_error = error(tmp_w, tmp_b)
        w, b = best_w, best_b
```

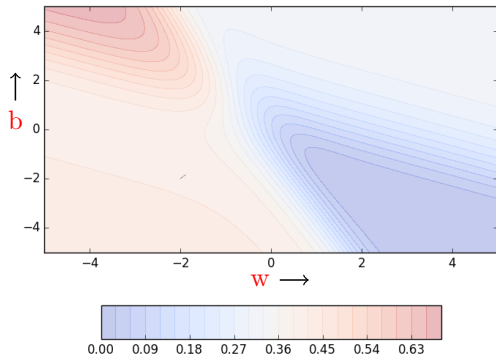

- In practice, often a line search is done to find a relatively better value of η
- Update w using different values of η
- Now retain that updated value of w which gives the lowest loss
- Essentially at each step we are trying to use the best η value from the available choices
- What's the flipside? We are doing many more computations in each step

```
def do_line_search_gradient_descent():  
    w, b, etas = init_w, init_b, [0.1, 0.5, 1.0, 5.0, 10.0]  
    for i in range(max_epochs):  
        dw, db = 0, 0  
        for x,y in zip(X, Y):  
            dw += grad_w(w, b, x, y)  
            db += grad_b(w, b, x, y)  
        min_error = 10000 #some large value  
        best_w, best_b = w, b  
        for eta in etas:  
            tmp_w = w - eta * dw  
            tmp_b = b - eta * db  
            if error(tmp_w, tmp_b) < min_error:  
                best_w = tmp_w  
                best_b = tmp_b  
                min_error = error(tmp_w, tmp_b)  
        w, b = best_w, best_b
```

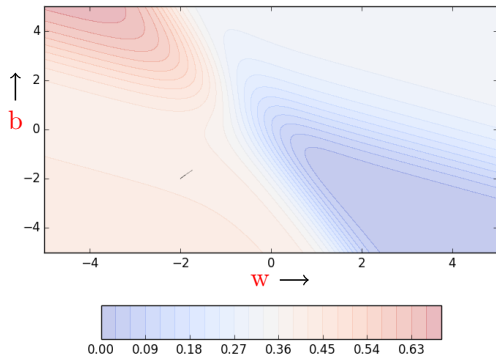
- In practice, often a line search is done to find a relatively better value of η
- Update w using different values of η
- Now retain that updated value of w which gives the lowest loss
- Essentially at each step we are trying to use the best η value from the available choices
- What's the flipside? We are doing many more computations in each step
- We will come back to this when we talk about second order optimization methods

```
def do_line_search_gradient_descent():  
    w, b, etas = init_w, init_b, [0.1, 0.5, 1.0, 5.0, 10.0]  
    for i in range(max_epochs):  
        dw, db = 0, 0  
        for x,y in zip(X, Y):  
            dw += grad_w(w, b, x, y)  
            db += grad_b(w, b, x, y)  
        min_error = 10000 #some large value  
        best_w, best_b = w, b  
        for eta in etas:  
            tmp_w = w - eta * dw  
            tmp_b = b - eta * db  
            if error(tmp_w, tmp_b) < min_error:  
                best_w = tmp_w  
                best_b = tmp_b  
                min_error = error(tmp_w, tmp_b)  
        w, b = best_w, best_b
```

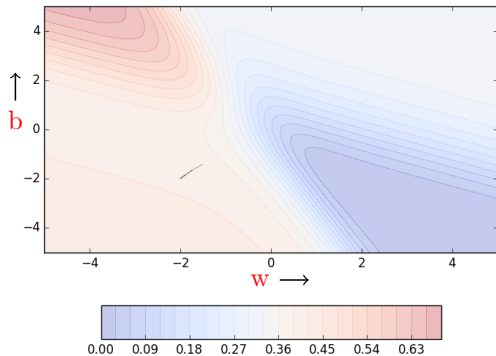
- Let us see line search in action



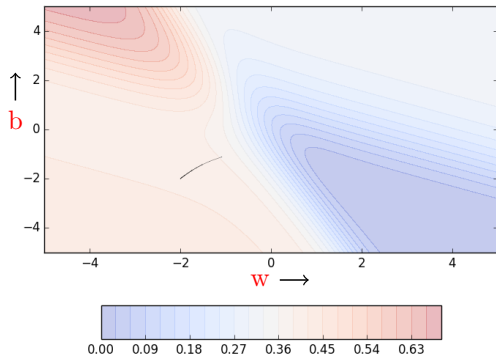
- Let us see line search in action



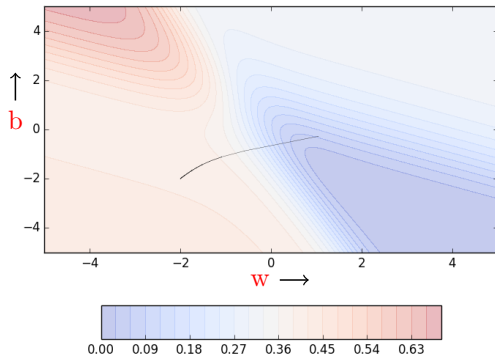
- Let us see line search in action



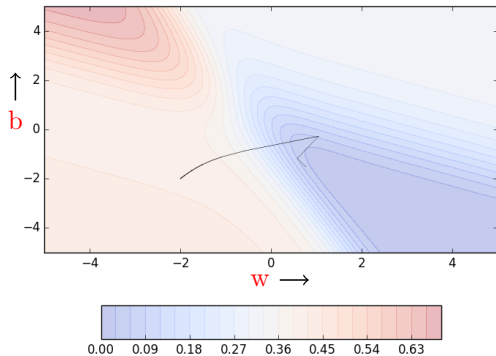
- Let us see line search in action



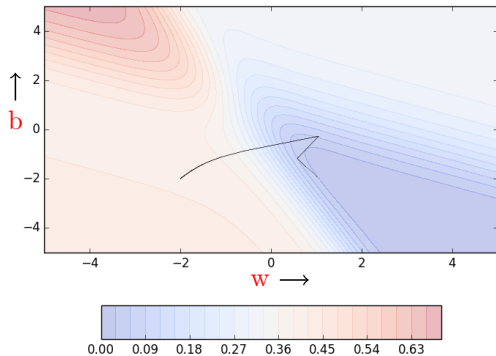
- Let us see line search in action



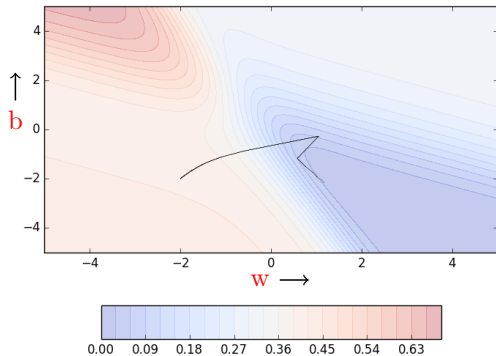
- Let us see line search in action



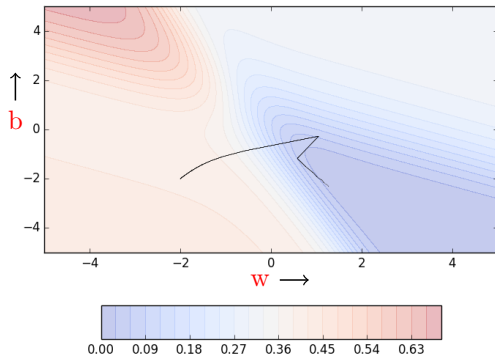
- Let us see line search in action



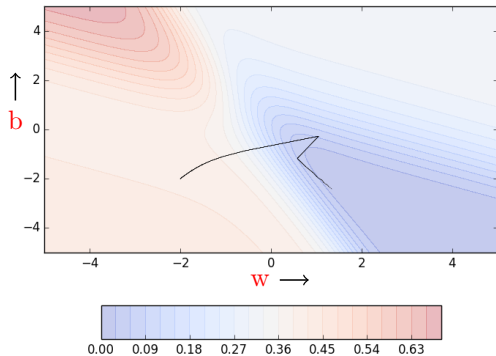
- Let us see line search in action



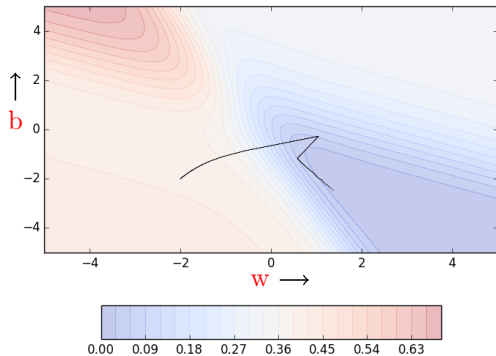
- Let us see line search in action



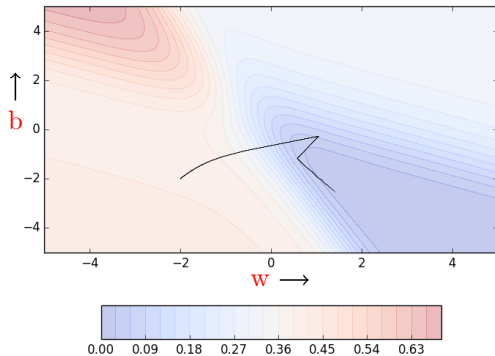
- Let us see line search in action



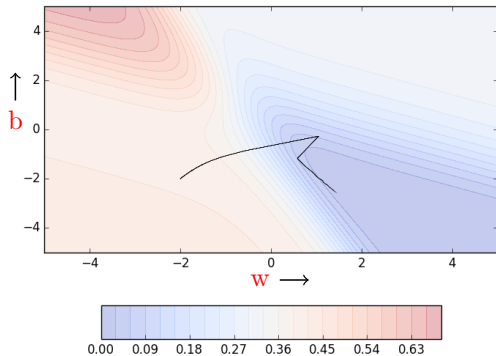
- Let us see line search in action



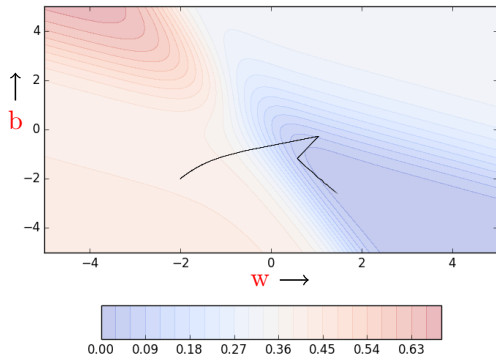
- Let us see line search in action



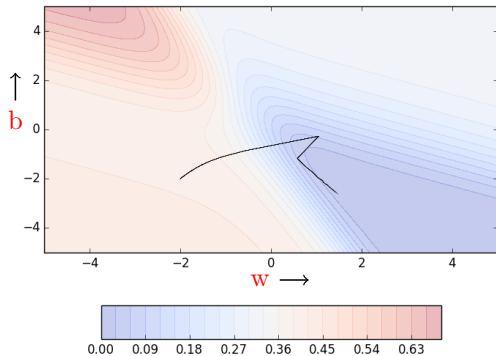
- Let us see line search in action



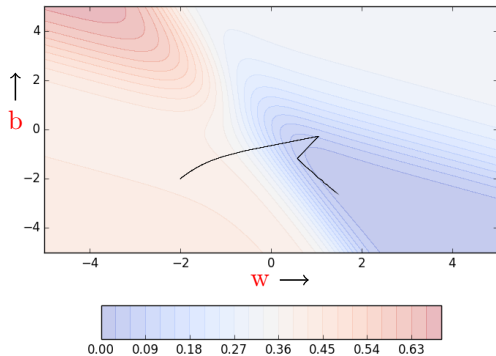
- Let us see line search in action



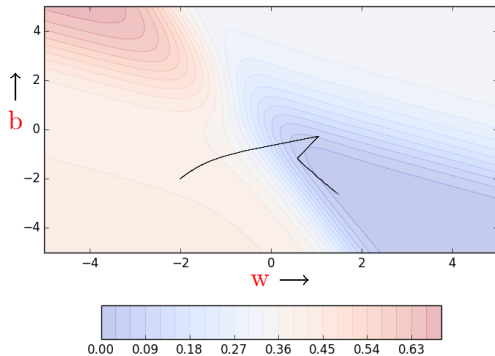
- Let us see line search in action



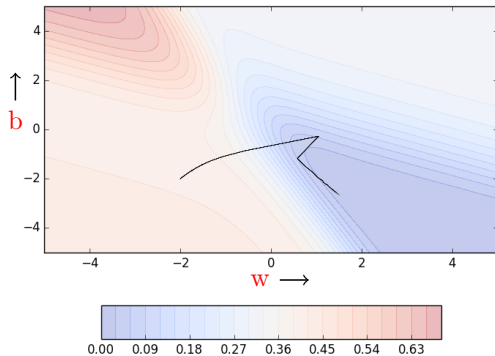
- Let us see line search in action



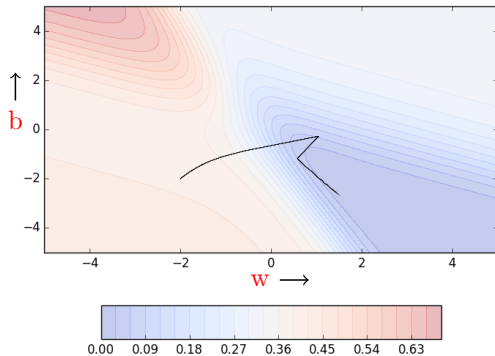
- Let us see line search in action
- Convergence is faster than vanilla gradient descent



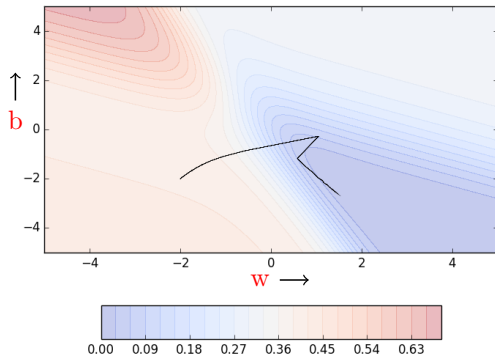
- Let us see line search in action
- Convergence is faster than vanilla gradient descent
- We see some oscillations,



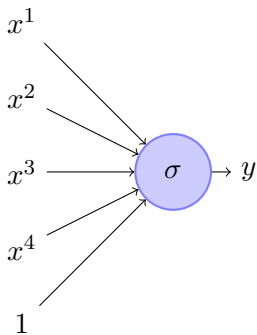
- Let us see line search in action
- Convergence is faster than vanilla gradient descent
- We see some oscillations, but note that these oscillations are different from what we see in momentum and NAG



- Let us see line search in action
- Convergence is faster than vanilla gradient descent
- We see some oscillations, but note that these oscillations are different from what we see in momentum and NAG



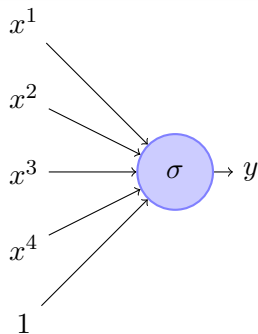
Module 5.9 : Gradient Descent with Adaptive Learning Rate



$$y = f(x) = \frac{1}{1 + e^{-(\mathbf{w} \cdot \mathbf{x} + b)}}$$

$$\mathbf{x} = \{x^1, x^2, x^3, x^4\}$$

$$\mathbf{w} = \{w^1, w^2, w^3, w^4\}$$

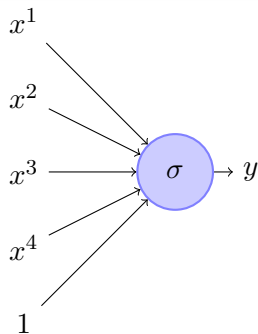


- Given this network, it should be easy to see that given a single point $(\mathbf{x}, y)$...

$$y = f(x) = \frac{1}{1+e^{-(\mathbf{w} \cdot \mathbf{x} + b)}}$$

$$\mathbf{x} = \{x^1, x^2, x^3, x^4\}$$

$$\mathbf{w} = \{w^1, w^2, w^3, w^4\}$$

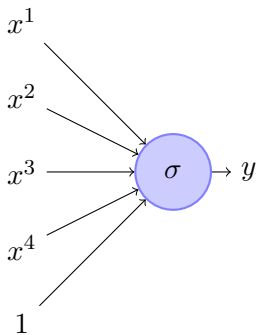


- Given this network, it should be easy to see that given a single point $(\mathbf{x}, y)$...
- $\nabla w^1 = (f(\mathbf{x}) - y) * f(\mathbf{x}) * (1 - f(\mathbf{x})) * x^1$
- $\nabla w^2 = (f(\mathbf{x}) - y) * f(\mathbf{x}) * (1 - f(\mathbf{x})) * x^2$... so on

$$y = f(x) = \frac{1}{1 + e^{-(\mathbf{w} \cdot \mathbf{x} + b)}}$$

$$\mathbf{x} = \{x^1, x^2, x^3, x^4\}$$

$$\mathbf{w} = \{w^1, w^2, w^3, w^4\}$$

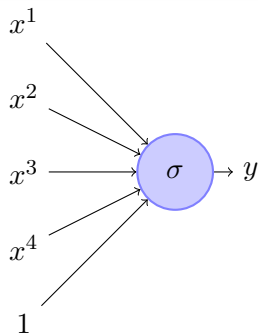


- Given this network, it should be easy to see that given a single point $(\mathbf{x}, y)$...
- $\nabla w^1 = (f(\mathbf{x}) - y) * f(\mathbf{x}) * (1 - f(\mathbf{x})) * x^1$
- $\nabla w^2 = (f(\mathbf{x}) - y) * f(\mathbf{x}) * (1 - f(\mathbf{x})) * x^2$... so on
- If there are n points, we can just sum the gradients over all the n points to get the total gradient

$$y = f(x) = \frac{1}{1 + e^{-(\mathbf{w} \cdot \mathbf{x} + b)}}$$

$$\mathbf{x} = \{x^1, x^2, x^3, x^4\}$$

$$\mathbf{w} = \{w^1, w^2, w^3, w^4\}$$

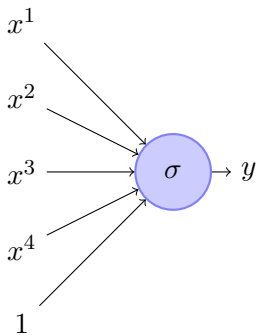


- Given this network, it should be easy to see that given a single point $(\mathbf{x}, y)$...
- $\nabla w^1 = (f(\mathbf{x}) - y) * f(\mathbf{x}) * (1 - f(\mathbf{x})) * x^1$
- $\nabla w^2 = (f(\mathbf{x}) - y) * f(\mathbf{x}) * (1 - f(\mathbf{x})) * x^2$... so on
- If there are n points, we can just sum the gradients over all the n points to get the total gradient
- What happens if the feature x^2 is very sparse?

$$y = f(x) = \frac{1}{1 + e^{-(\mathbf{w} \cdot \mathbf{x} + b)}}$$

$$\mathbf{x} = \{x^1, x^2, x^3, x^4\}$$

$$\mathbf{w} = \{w^1, w^2, w^3, w^4\}$$

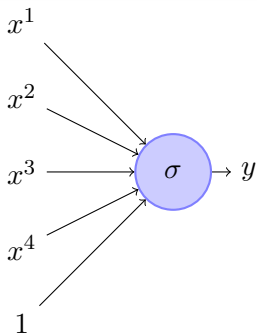


- Given this network, it should be easy to see that given a single point $(\mathbf{x}, y)$...
- $\nabla w^1 = (f(\mathbf{x}) - y) * f(\mathbf{x}) * (1 - f(\mathbf{x})) * x^1$
- $\nabla w^2 = (f(\mathbf{x}) - y) * f(\mathbf{x}) * (1 - f(\mathbf{x})) * x^2$... so on
- If there are n points, we can just sum the gradients over all the n points to get the total gradient
- What happens if the feature x^2 is very sparse? (*i.e.*, if its value is 0 for most inputs)

$$y = f(x) = \frac{1}{1 + e^{-(\mathbf{w} \cdot \mathbf{x} + b)}}$$

$$\mathbf{x} = \{x^1, x^2, x^3, x^4\}$$

$$\mathbf{w} = \{w^1, w^2, w^3, w^4\}$$

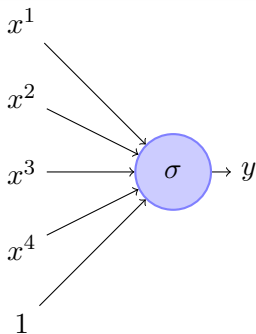


$$y = f(x) = \frac{1}{1 + e^{-(\mathbf{w} \cdot \mathbf{x} + b)}}$$

$$\mathbf{x} = \{x^1, x^2, x^3, x^4\}$$

$$\mathbf{w} = \{w^1, w^2, w^3, w^4\}$$

- Given this network, it should be easy to see that given a single point $(\mathbf{x}, y)$...
- $\nabla w^1 = (f(\mathbf{x}) - y) * f(\mathbf{x}) * (1 - f(\mathbf{x})) * x^1$
- $\nabla w^2 = (f(\mathbf{x}) - y) * f(\mathbf{x}) * (1 - f(\mathbf{x})) * x^2$... so on
- If there are n points, we can just sum the gradients over all the n points to get the total gradient
- What happens if the feature x^2 is very sparse? (*i.e.*, if its value is 0 for most inputs)
- ∇w^2 will be 0 for most inputs (see formula) and hence w^2 will not get enough updates

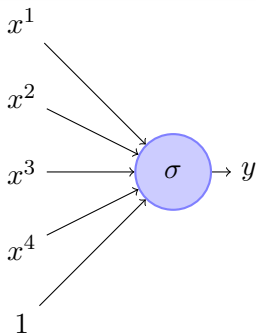


$$y = f(x) = \frac{1}{1 + e^{-(\mathbf{w} \cdot \mathbf{x} + b)}}$$

$$\mathbf{x} = \{x^1, x^2, x^3, x^4\}$$

$$\mathbf{w} = \{w^1, w^2, w^3, w^4\}$$

- Given this network, it should be easy to see that given a single point $(\mathbf{x}, y)$...
- $\nabla w^1 = (f(\mathbf{x}) - y) * f(\mathbf{x}) * (1 - f(\mathbf{x})) * x^1$
- $\nabla w^2 = (f(\mathbf{x}) - y) * f(\mathbf{x}) * (1 - f(\mathbf{x})) * x^2$... so on
- If there are n points, we can just sum the gradients over all the n points to get the total gradient
- What happens if the feature x^2 is very sparse? (*i.e.*, if its value is 0 for most inputs)
- ∇w^2 will be 0 for most inputs (see formula) and hence w^2 will not get enough updates
- If x^2 happens to be sparse as well as important we would want to take the updates to w^2 more seriously



$$y = f(x) = \frac{1}{1 + e^{-(\mathbf{w} \cdot \mathbf{x} + b)}}$$

$$\mathbf{x} = \{x^1, x^2, x^3, x^4\}$$

$$\mathbf{w} = \{w^1, w^2, w^3, w^4\}$$

- Given this network, it should be easy to see that given a single point $(\mathbf{x}, y)$...
- $\nabla w^1 = (f(\mathbf{x}) - y) * f(\mathbf{x}) * (1 - f(\mathbf{x})) * x^1$
- $\nabla w^2 = (f(\mathbf{x}) - y) * f(\mathbf{x}) * (1 - f(\mathbf{x})) * x^2$... so on
- If there are n points, we can just sum the gradients over all the n points to get the total gradient
- What happens if the feature x^2 is very sparse? (*i.e.*, if its value is 0 for most inputs)
- ∇w^2 will be 0 for most inputs (see formula) and hence w^2 will not get enough updates
- If x^2 happens to be sparse as well as important we would want to take the updates to w^2 more seriously
- Can we have a different learning rate for each parameter which takes care of the frequency of features ?

Intuition

- Decay the learning rate for parameters in proportion to their update history

Intuition

- Decay the learning rate for parameters in proportion to their update history (more updates means more decay)

Intuition

- Decay the learning rate for parameters in proportion to their update history (more updates means more decay)

Update rule for Adagrad

$$v_t = v_{t-1} + (\nabla w_t)^2$$
$$w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t + \epsilon}} * \nabla w_t$$

... and a similar set of equations for b_t

- To see this in action we need to first create some data where one of the features is sparse

```
def do_adagrad():  
    w, b, eta = init_w, init_b, 0.1  
    v_w, v_b, eps = 0, 0, 1e-8  
    for i in range(max_epochs):  
        dw, db = 0, 0  
        for x,y in zip(X, Y):  
            dw += grad_w(w, b, x, y)  
            db += grad_b(w, b, x, y)  
  
        v_w = v_w + dw**2  
        v_b = v_b + db**2  
  
        w = w - (eta / np.sqrt(v_w + eps)) * dw  
        b = b - (eta / np.sqrt(v_b + eps)) * db
```

- To see this in action we need to first create some data where one of the features is sparse
- How would we do this in our toy network ?

```
def do_adagrad():  
    w, b, eta = init_w, init_b, 0.1  
    v_w, v_b, eps = 0, 0, 1e-8  
    for i in range(max_epochs):  
        dw, db = 0, 0  
        for x,y in zip(X, Y):  
            dw += grad_w(w, b, x, y)  
            db += grad_b(w, b, x, y)  
  
        v_w = v_w + dw**2  
        v_b = v_b + db**2  
  
        w = w - (eta / np.sqrt(v_w + eps)) * dw  
        b = b - (eta / np.sqrt(v_b + eps)) * db
```

- To see this in action we need to first create some data where one of the features is sparse
- How would we do this in our toy network ?
Take some time to think about it

```
def do_adagrad():  
    w, b, eta = init_w, init_b, 0.1  
    v_w, v_b, eps = 0, 0, 1e-8  
    for i in range(max_epochs):  
        dw, db = 0, 0  
        for x,y in zip(X, Y):  
            dw += grad_w(w, b, x, y)  
            db += grad_b(w, b, x, y)  
  
        v_w = v_w + dw**2  
        v_b = v_b + db**2  
  
        w = w - (eta / np.sqrt(v_w + eps)) * dw  
        b = b - (eta / np.sqrt(v_b + eps)) * db
```

- To see this in action we need to first create some data where one of the features is sparse
- How would we do this in our toy network ?
Take some time to think about it
- Well, our network has just two parameters (w and b).

```
def do_adagrad():  
    w, b, eta = init_w, init_b, 0.1  
    v_w, v_b, eps = 0, 0, 1e-8  
    for i in range(max_epochs):  
        dw, db = 0, 0  
        for x,y in zip(X, Y):  
            dw += grad_w(w, b, x, y)  
            db += grad_b(w, b, x, y)  
  
        v_w = v_w + dw**2  
        v_b = v_b + db**2  
  
        w = w - (eta / np.sqrt(v_w + eps)) * dw  
        b = b - (eta / np.sqrt(v_b + eps)) * db
```

- To see this in action we need to first create some data where one of the features is sparse
- How would we do this in our toy network ?
Take some time to think about it
- Well, our network has just two parameters (w and b). Of these, the input/feature corresponding to b is always on

```
def do_adagrad():  
    w, b, eta = init_w, init_b, 0.1  
    v_w, v_b, eps = 0, 0, 1e-8  
    for i in range(max_epochs):  
        dw, db = 0, 0  
        for x,y in zip(X, Y):  
            dw += grad_w(w, b, x, y)  
            db += grad_b(w, b, x, y)  
  
        v_w = v_w + dw**2  
        v_b = v_b + db**2  
  
        w = w - (eta / np.sqrt(v_w + eps)) * dw  
        b = b - (eta / np.sqrt(v_b + eps)) * db
```


- To see this in action we need to first create some data where one of the features is sparse
- How would we do this in our toy network ?
Take some time to think about it
- Well, our network has just two parameters (w and b). Of these, the input/feature corresponding to b is always on (so can't really make it sparse)

```
def do_adagrad():
    w, b, eta = init_w, init_b, 0.1
    v_w, v_b, eps = 0, 0, 1e-8
    for i in range(max_epochs):
        dw, db = 0, 0
        for x,y in zip(X, Y):
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = v_w + dw**2
        v_b = v_b + db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db
```

- To see this in action we need to first create some data where one of the features is sparse
- How would we do this in our toy network ?
Take some time to think about it
- Well, our network has just two parameters (w and b). Of these, the input/feature corresponding to b is always on (so can't really make it sparse)
- The only option is to make x sparse

```
def do_adagrad():
    w, b, eta = init_w, init_b, 0.1
    v_w, v_b, eps = 0, 0, 1e-8
    for i in range(max_epochs):
        dw, db = 0, 0
        for x, y in zip(X, Y):
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = v_w + dw**2
        v_b = v_b + db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db
```

- To see this in action we need to first create some data where one of the features is sparse
- How would we do this in our toy network ? Take some time to think about it
- Well, our network has just two parameters (w and b). Of these, the input/feature corresponding to b is always on (so can't really make it sparse)
- The only option is to make x sparse
- **Solution:** We created 100 random (x, y) pairs and then for roughly 80% of these pairs we set x to 0

```
def do_adagrad():
    w, b, eta = init_w, init_b, 0.1
    v_w, v_b, eps = 0, 0, 1e-8
    for i in range(max_epochs):
        dw, db = 0, 0
        for x, y in zip(X, Y):
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = v_w + dw**2
        v_b = v_b + db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db
```

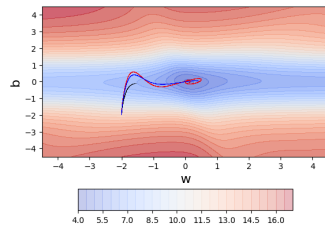
- To see this in action we need to first create some data where one of the features is sparse
- How would we do this in our toy network ?
Take some time to think about it
- Well, our network has just two parameters (w and b). Of these, the input/feature corresponding to b is always on (so can't really make it sparse)
- The only option is to make x sparse
- **Solution:** We created 100 random (x, y) pairs and then for roughly 80% of these pairs we set x to 0 thereby, making the feature for w sparse

```
def do_adagrad():
    w, b, eta = init_w, init_b, 0.1
    v_w, v_b, eps = 0, 0, 1e-8
    for i in range(max_epochs):
        dw, db = 0, 0
        for x, y in zip(X, Y):
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

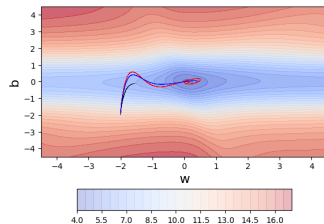
        v_w = v_w + dw**2
        v_b = v_b + db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db
```

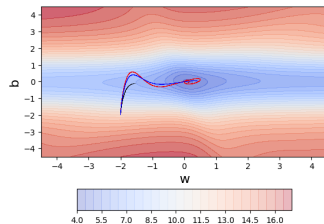
- GD (black), momentum (red) and NAG (blue)



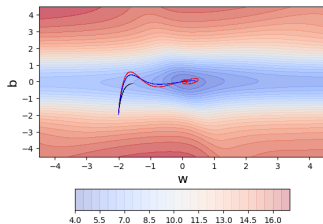
- GD (black), momentum (red) and NAG (blue)
- There is something interesting that these 3 algorithms are doing for this dataset.



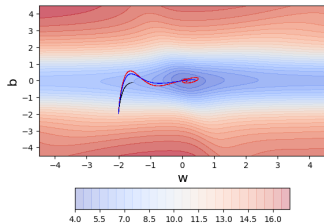
- GD (black), momentum (red) and NAG (blue)
- There is something interesting that these 3 algorithms are doing for this dataset. Can you spot it?



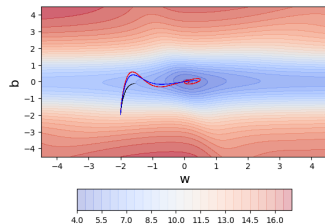
- GD (black), momentum (red) and NAG (blue)
- There is something interesting that these 3 algorithms are doing for this dataset. Can you spot it?
- Initially, all three algorithms are moving mainly along the vertical (b) axis and there is very little movement along the horizontal (w) axis



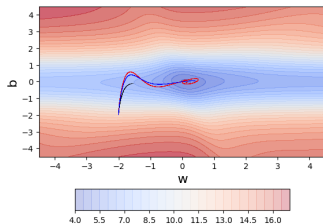
- GD (black), momentum (red) and NAG (blue)
- There is something interesting that these 3 algorithms are doing for this dataset. Can you spot it?
- Initially, all three algorithms are moving mainly along the vertical (b) axis and there is very little movement along the horizontal (w) axis
- Why?



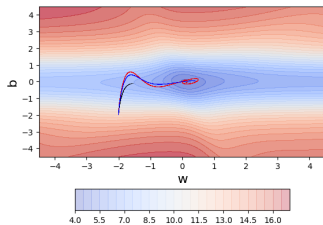
- GD (black), momentum (red) and NAG (blue)
- There is something interesting that these 3 algorithms are doing for this dataset. Can you spot it?
- Initially, all three algorithms are moving mainly along the vertical (b) axis and there is very little movement along the horizontal (w) axis
- Why? Because in our data, the feature corresponding to w is sparse and hence w undergoes very few updates



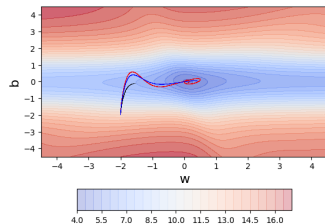
- GD (black), momentum (red) and NAG (blue)
- There is something interesting that these 3 algorithms are doing for this dataset. Can you spot it?
- Initially, all three algorithms are moving mainly along the vertical (b) axis and there is very little movement along the horizontal (w) axis
- Why? Because in our data, the feature corresponding to w is sparse and hence w undergoes very few updates ...on the other hand b is very dense and undergoes many updates



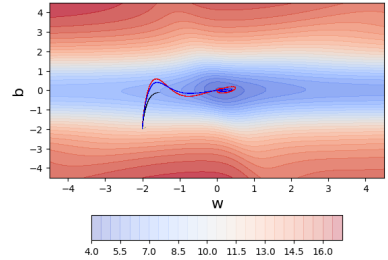
- GD (black), momentum (red) and NAG (blue)
- There is something interesting that these 3 algorithms are doing for this dataset. Can you spot it?
- Initially, all three algorithms are moving mainly along the vertical (b) axis and there is very little movement along the horizontal (w) axis
- Why? Because in our data, the feature corresponding to w is sparse and hence w undergoes very few updates ...on the other hand b is very dense and undergoes many updates
- Such sparsity is very common in large neural networks containing 1000s of input features and hence we need to address it

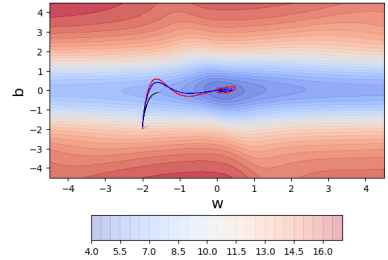


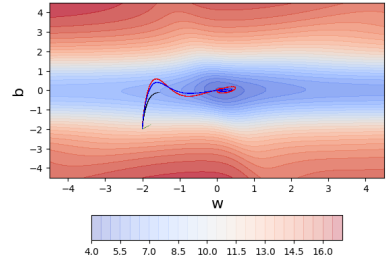
- GD (black), momentum (red) and NAG (blue)
- There is something interesting that these 3 algorithms are doing for this dataset. Can you spot it?
- Initially, all three algorithms are moving mainly along the vertical (b) axis and there is very little movement along the horizontal (w) axis
- Why? Because in our data, the feature corresponding to w is sparse and hence w undergoes very few updates ...on the other hand b is very dense and undergoes many updates
- Such sparsity is very common in large neural networks containing 1000s of input features and hence we need to address it

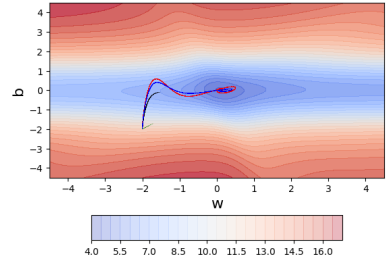


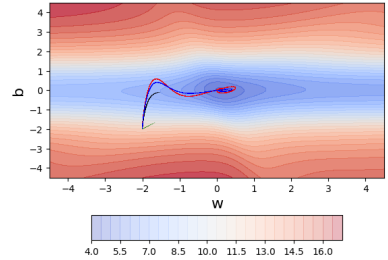
- Let's see what Adagrad does....

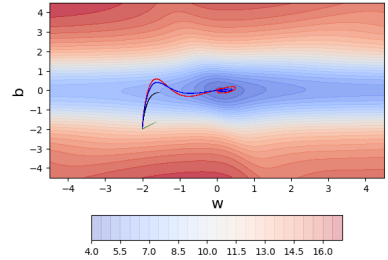


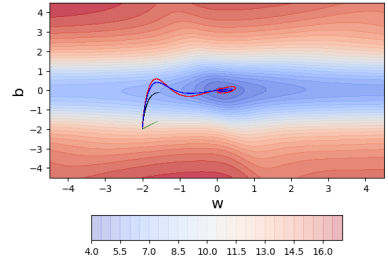


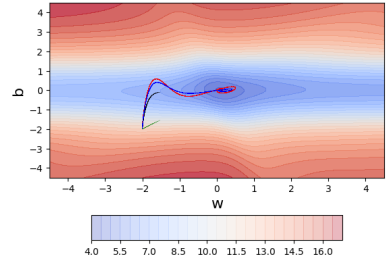


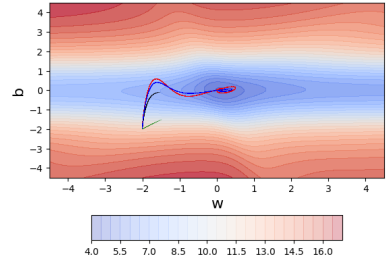


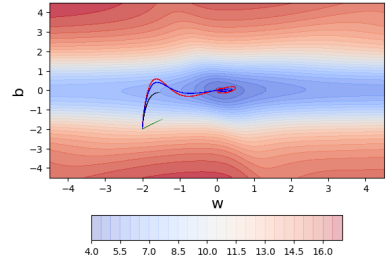


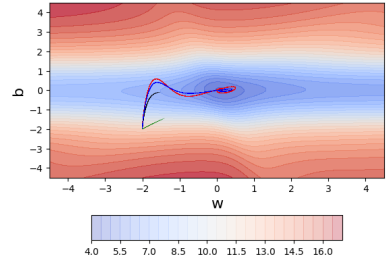


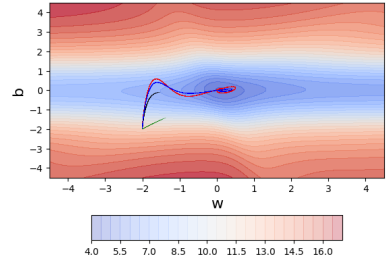


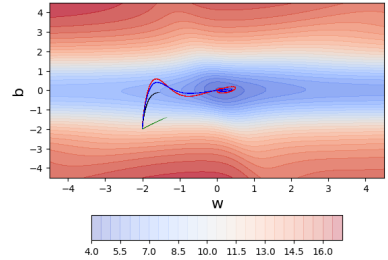


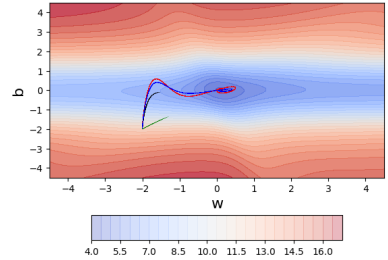


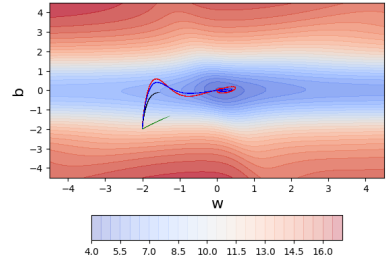


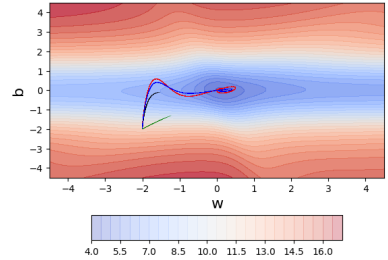


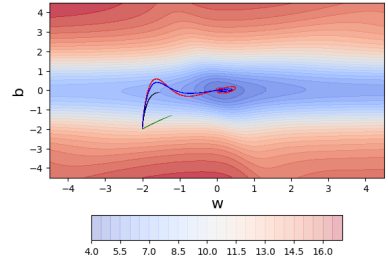


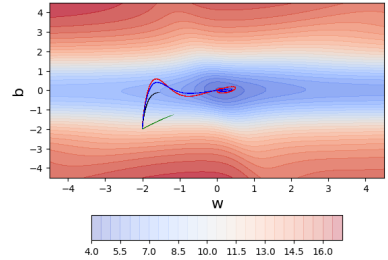


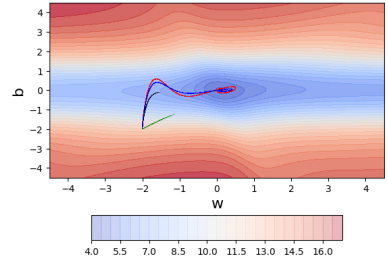


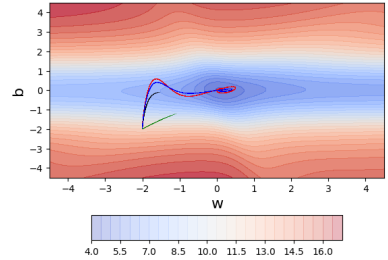


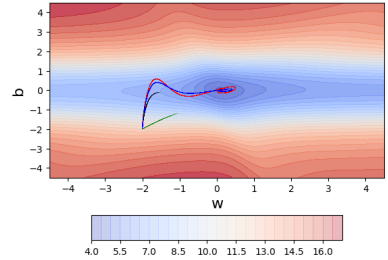


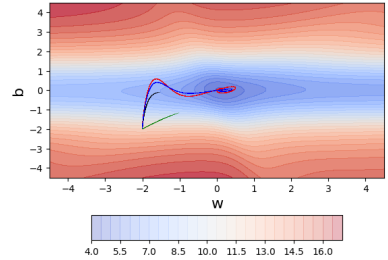


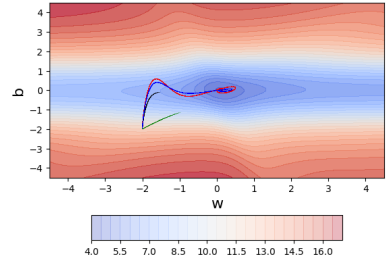


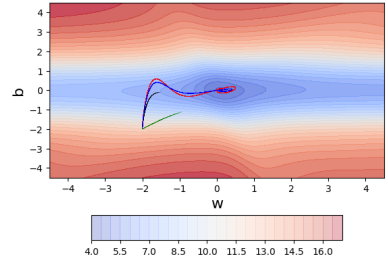


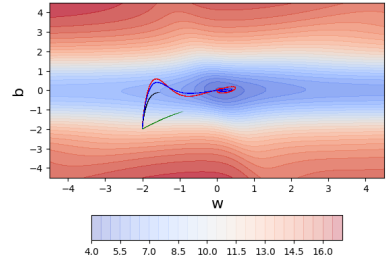


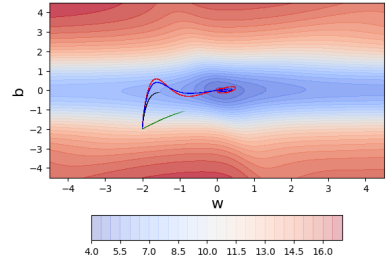


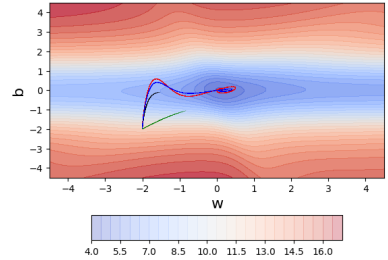


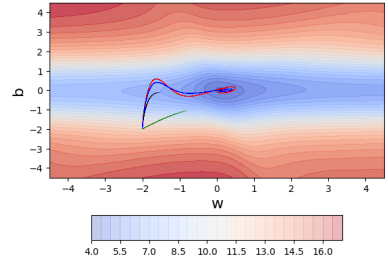


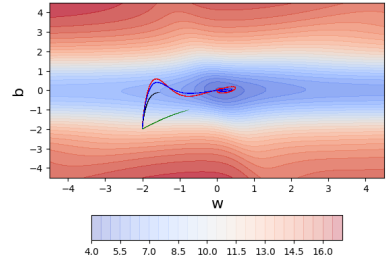


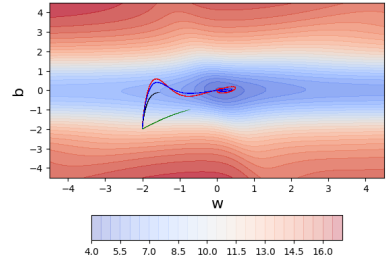


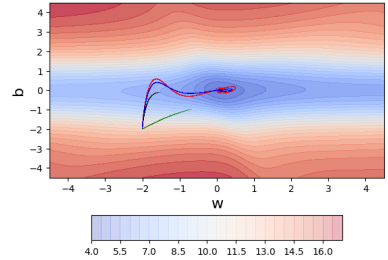


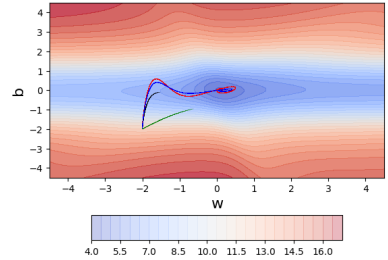


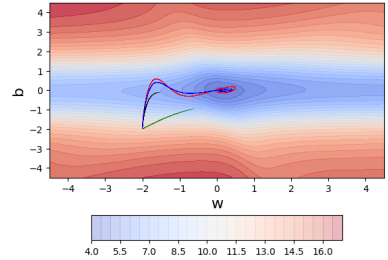


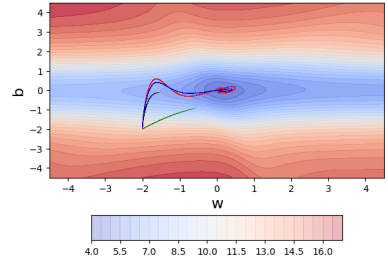


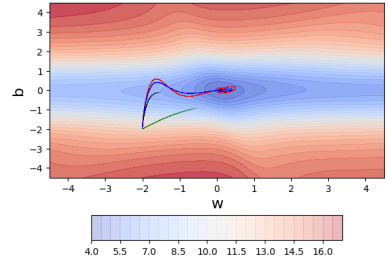


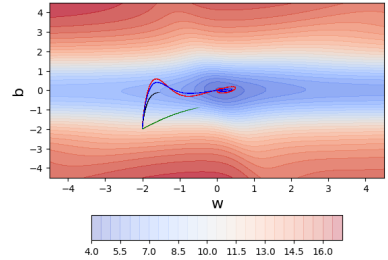


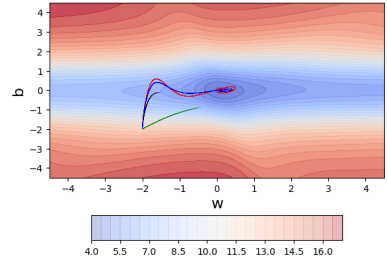


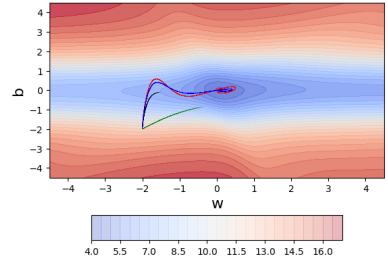


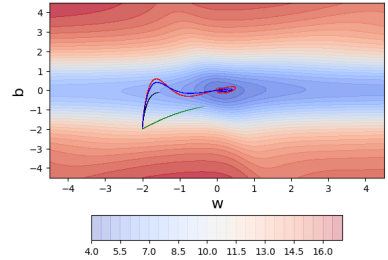


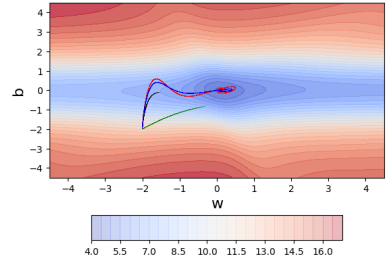


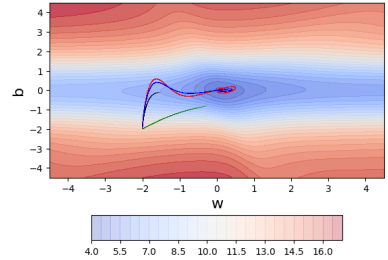


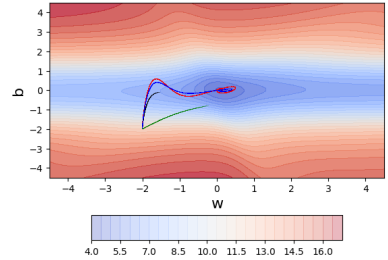


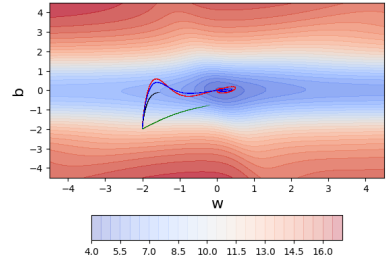


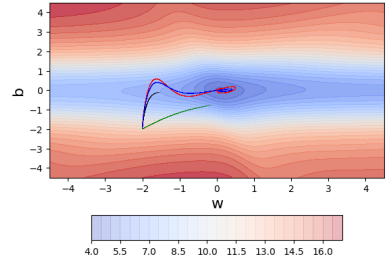


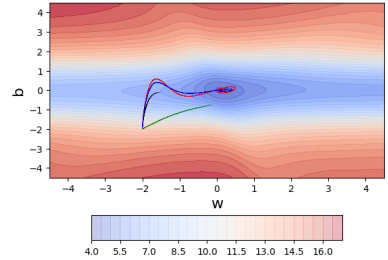


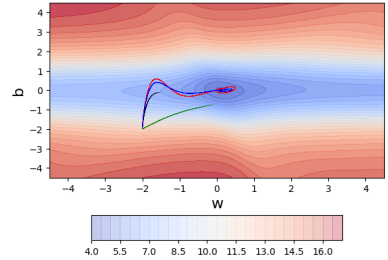


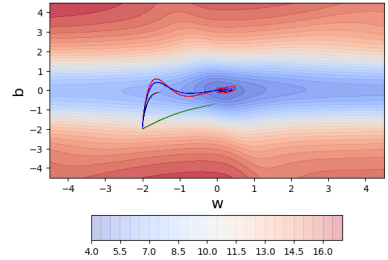


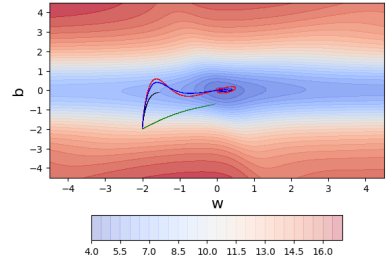


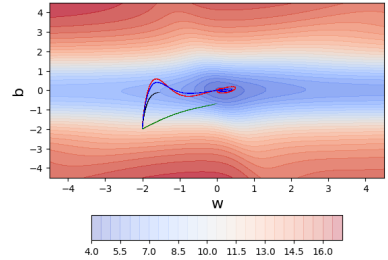


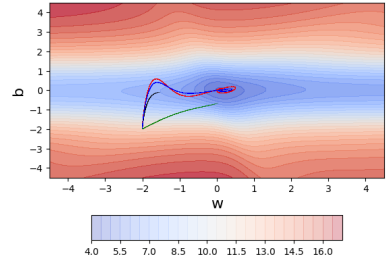


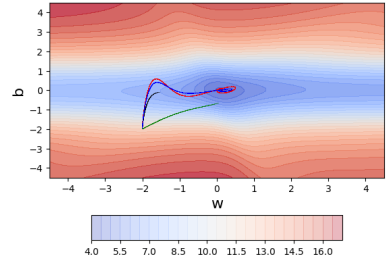


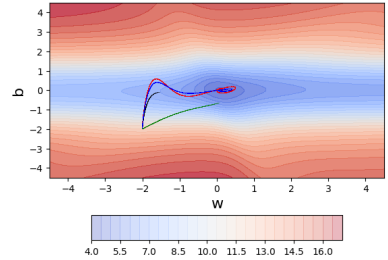


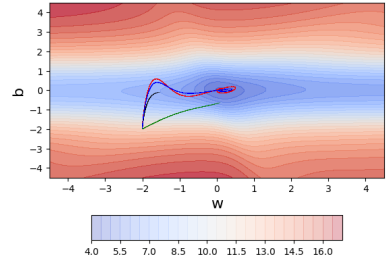


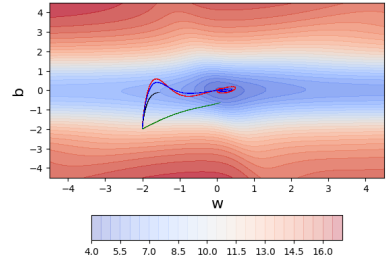


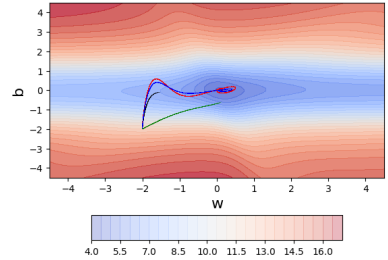


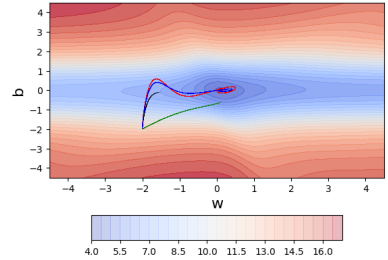


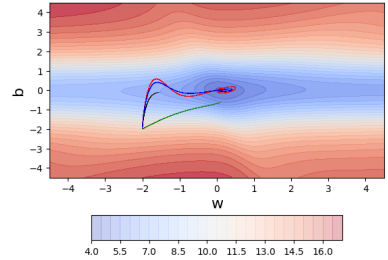


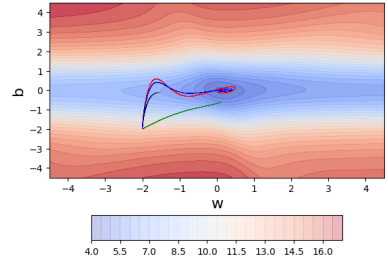


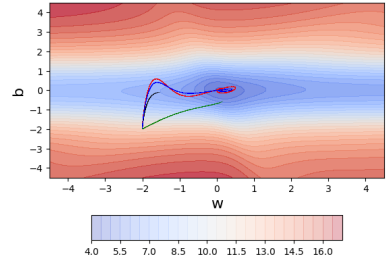


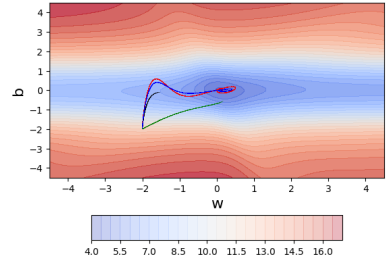


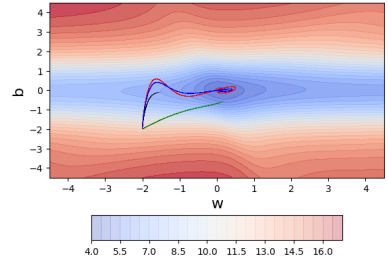


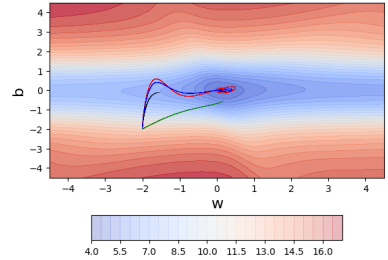


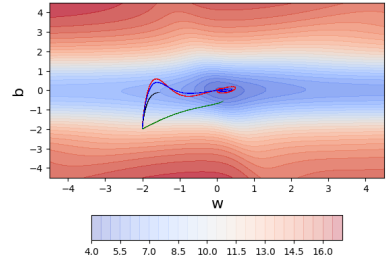


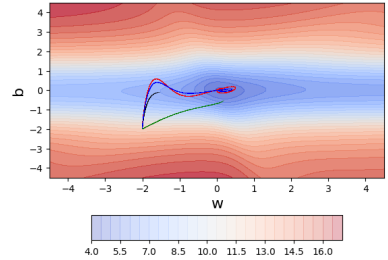


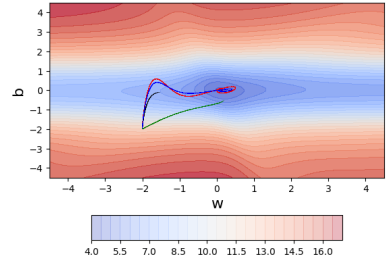


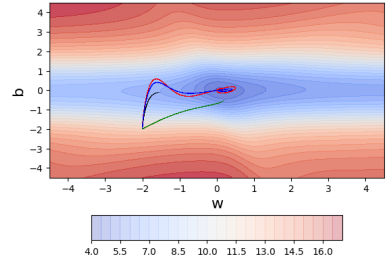


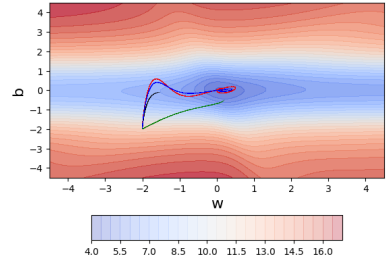


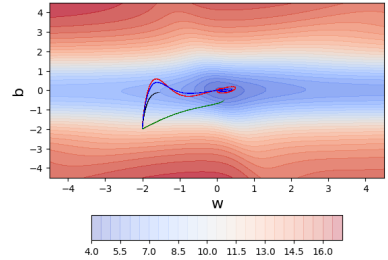


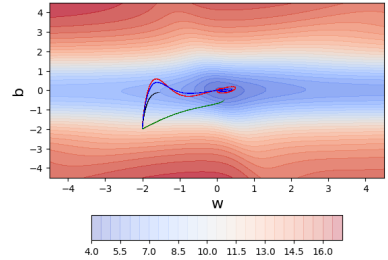


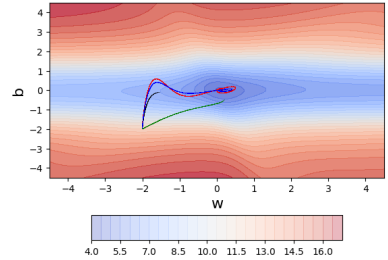


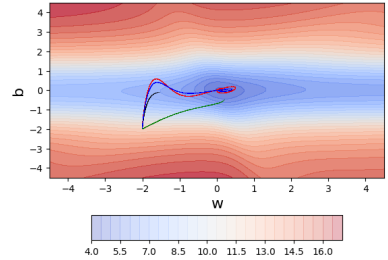


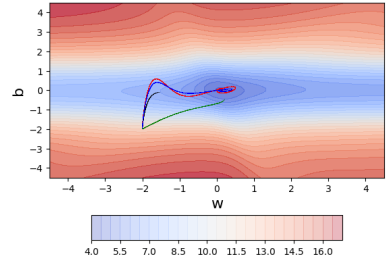


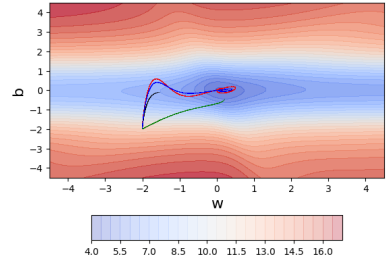


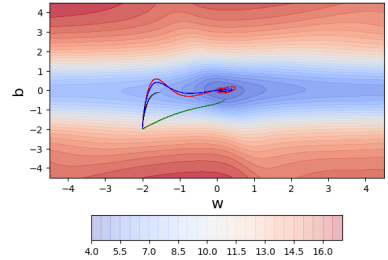


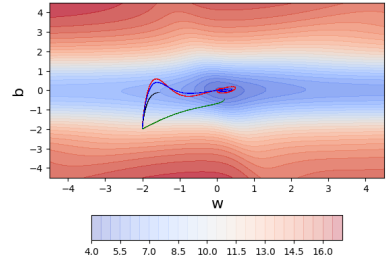


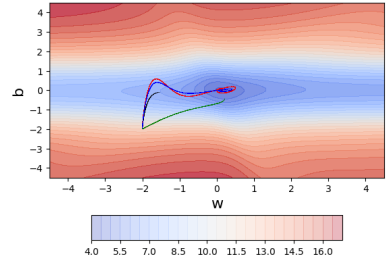


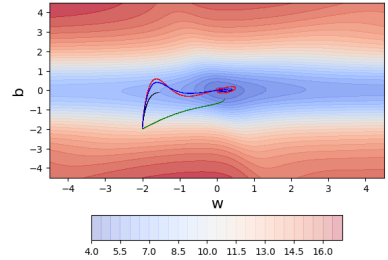


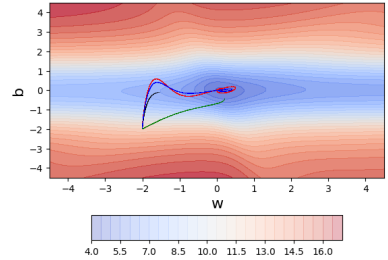


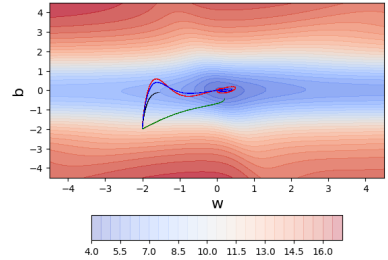


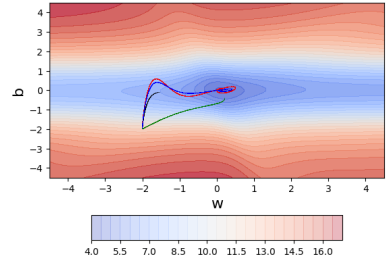


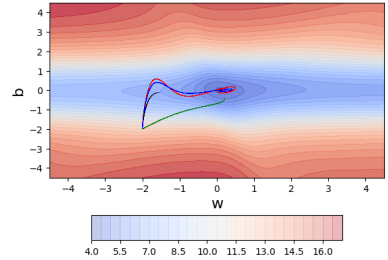


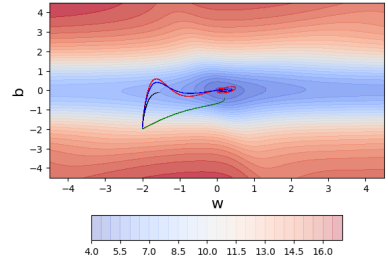


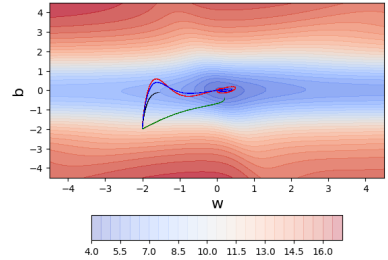


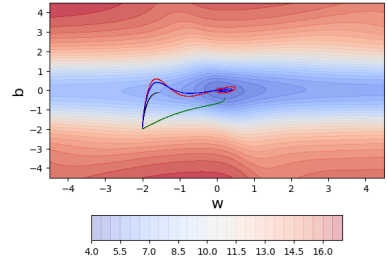


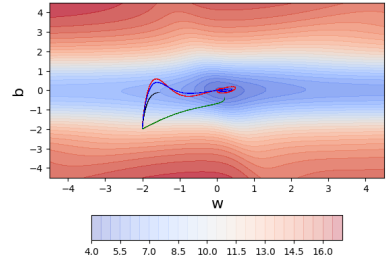


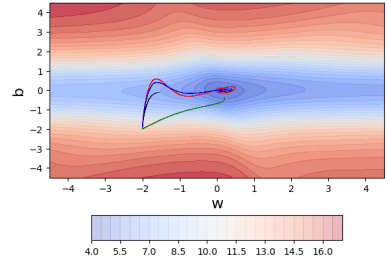


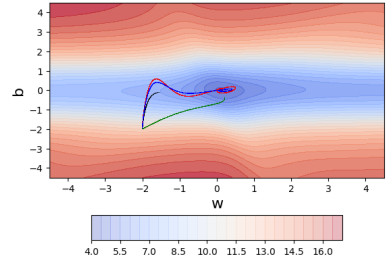


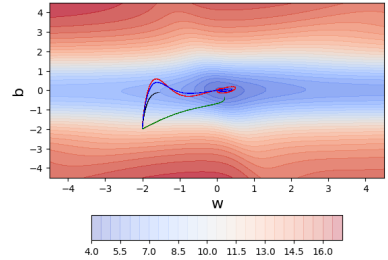


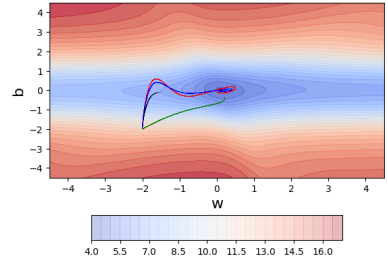


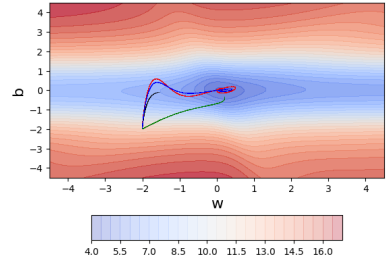




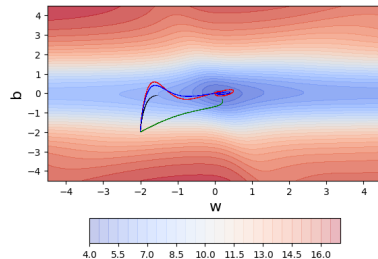




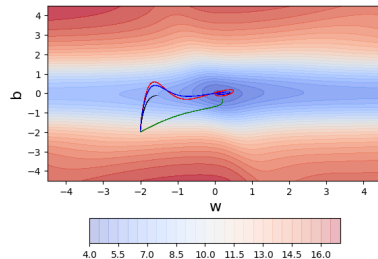




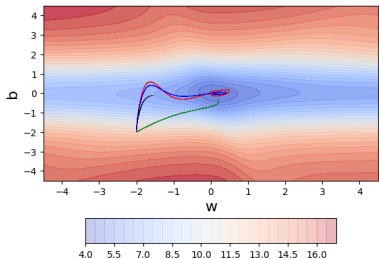
- By using a parameter specific learning rate it ensures that despite sparsity w gets a higher learning rate and hence larger updates



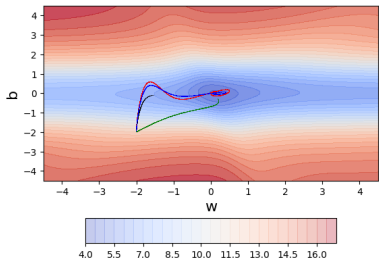
- By using a parameter specific learning rate it ensures that despite sparsity w gets a higher learning rate and hence larger updates
- Further, it also ensures that if b undergoes a lot of updates its effective learning rate decreases because of the growing denominator



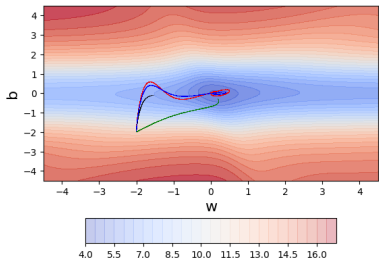
- By using a parameter specific learning rate it ensures that despite sparsity w gets a higher learning rate and hence larger updates
- Further, it also ensures that if b undergoes a lot of updates its effective learning rate decreases because of the growing denominator
- In practice, this does not work so well if we remove the square root from the denominator



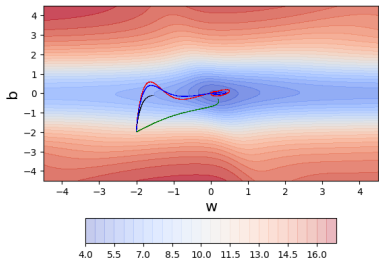
- By using a parameter specific learning rate it ensures that despite sparsity w gets a higher learning rate and hence larger updates
- Further, it also ensures that if b undergoes a lot of updates its effective learning rate decreases because of the growing denominator
- In practice, this does not work so well if we remove the square root from the denominator (something to ponder about)



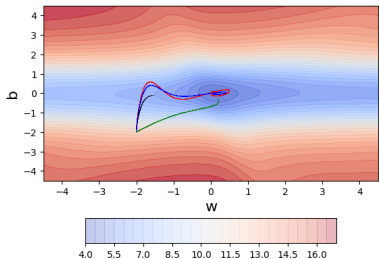
- By using a parameter specific learning rate it ensures that despite sparsity w gets a higher learning rate and hence larger updates
- Further, it also ensures that if b undergoes a lot of updates its effective learning rate decreases because of the growing denominator
- In practice, this does not work so well if we remove the square root from the denominator (something to ponder about)
- What's the flipside?



- By using a parameter specific learning rate it ensures that despite sparsity w gets a higher learning rate and hence larger updates
- Further, it also ensures that if b undergoes a lot of updates its effective learning rate decreases because of the growing denominator
- In practice, this does not work so well if we remove the square root from the denominator (something to ponder about)
- What's the flipside? over time the effective learning rate for b will decay to an extent that there will be no further updates to b



- By using a parameter specific learning rate it ensures that despite sparsity w gets a higher learning rate and hence larger updates
- Further, it also ensures that if b undergoes a lot of updates its effective learning rate decreases because of the growing denominator
- In practice, this does not work so well if we remove the square root from the denominator (something to ponder about)
- What's the flipside? over time the effective learning rate for b will decay to an extent that there will be no further updates to b
- Can we avoid this?



Intuition

- Adagrad decays the learning rate very aggressively (as the denominator grows)

Intuition

- Adagrad decays the learning rate very aggressively (as the denominator grows)
- As a result after a while the frequent parameters will start receiving very small updates because of the decayed learning rate

Intuition

- Adagrad decays the learning rate very aggressively (as the denominator grows)
- As a result after a while the frequent parameters will start receiving very small updates because of the decayed learning rate
- To avoid this why not decay the denominator and prevent its rapid growth

Intuition

- Adagrad decays the learning rate very aggressively (as the denominator grows)
- As a result after a while the frequent parameters will start receiving very small updates because of the decayed learning rate
- To avoid this why not decay the denominator and prevent its rapid growth

Update rule for RMSProp

$$v_t = \beta * v_{t-1} + (1 - \beta)(\nabla w_t)^2$$
$$w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t} + \epsilon} * \nabla w_t$$

... and a similar set of equations for b_t

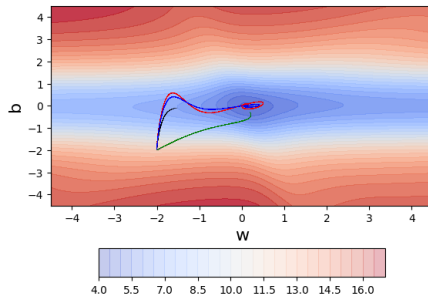
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



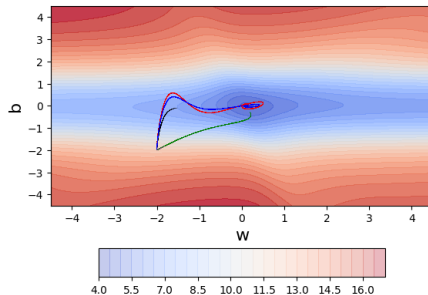
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



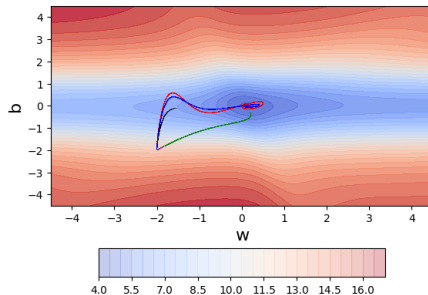
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



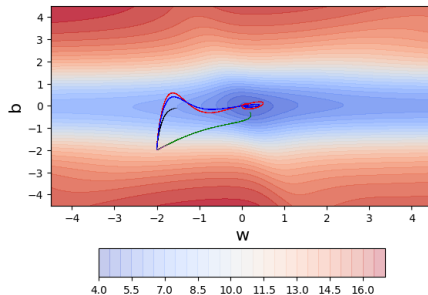
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



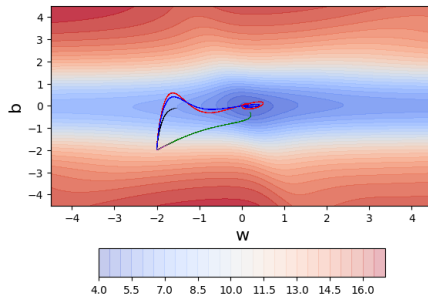
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



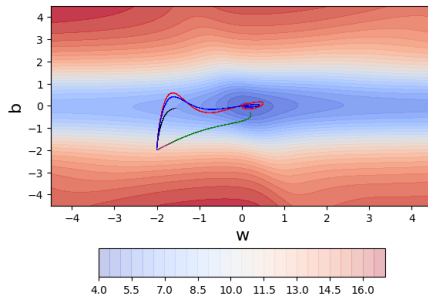
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



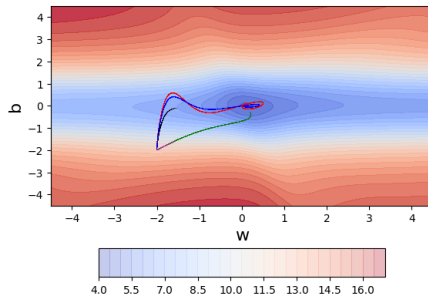

```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



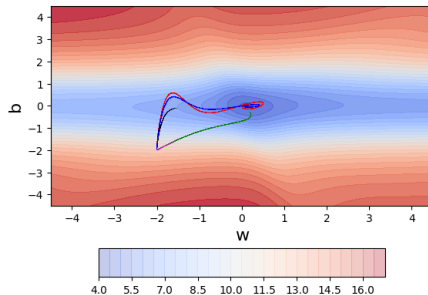
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



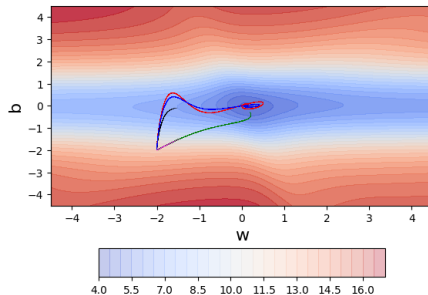
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



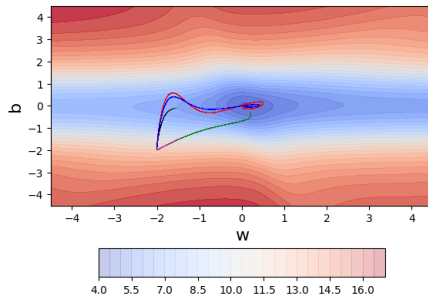
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

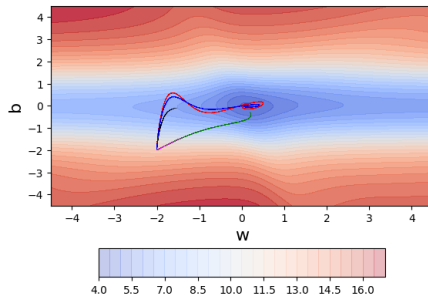
```



```
def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db
```



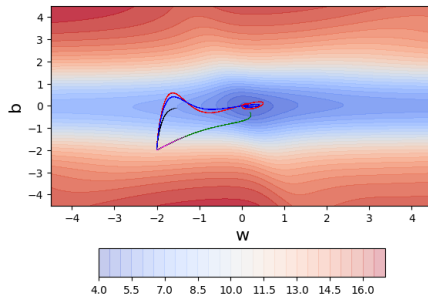
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



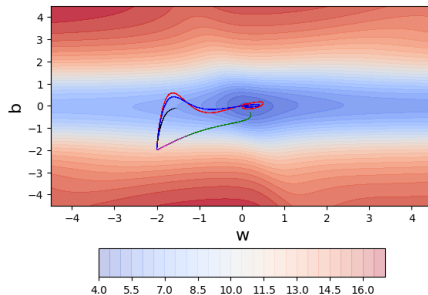
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



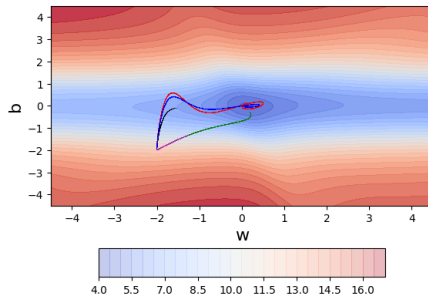
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



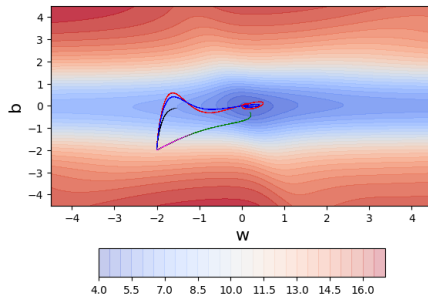

```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

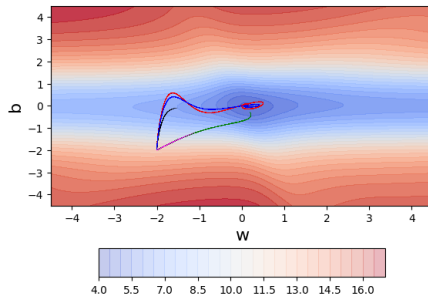
```



```
def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db
```



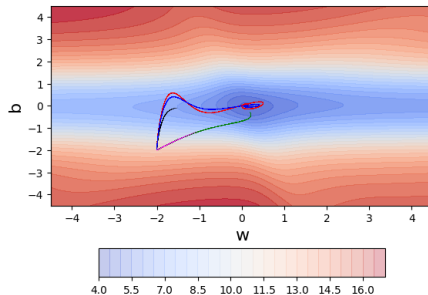
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



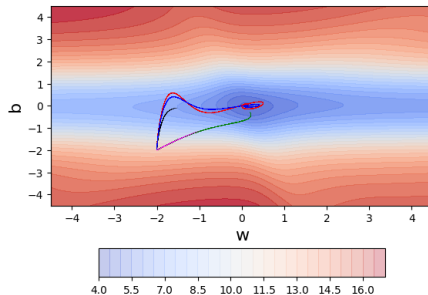
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



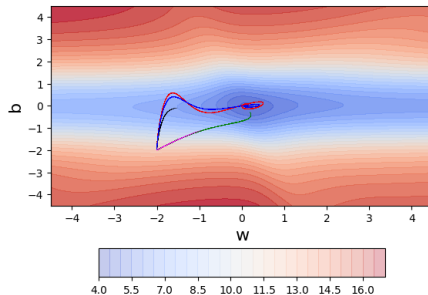
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



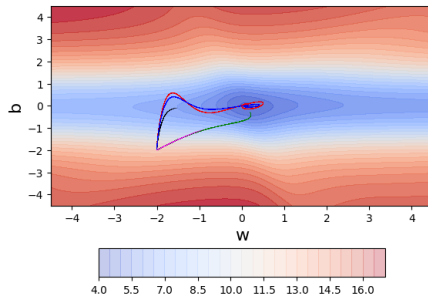
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



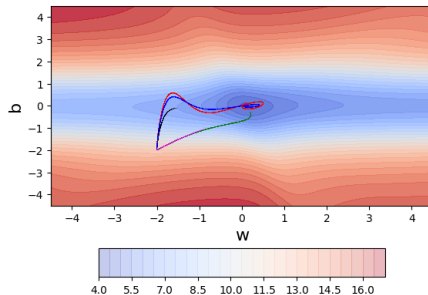
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



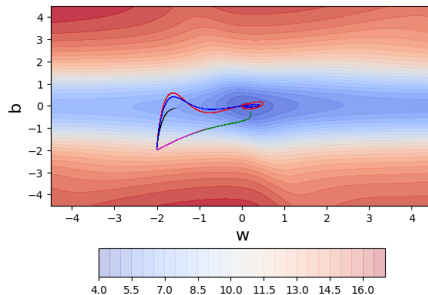
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



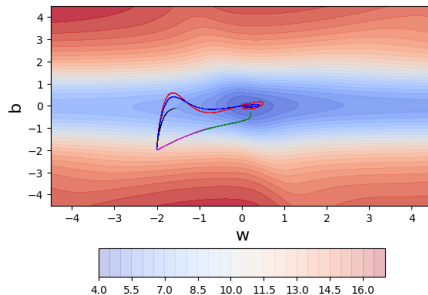

```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



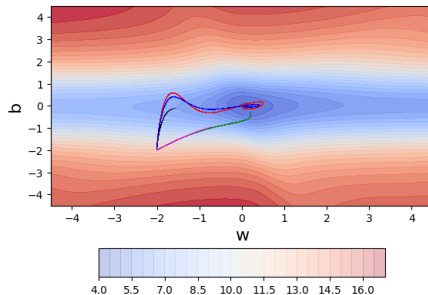
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



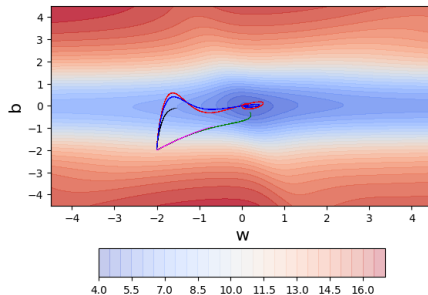
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



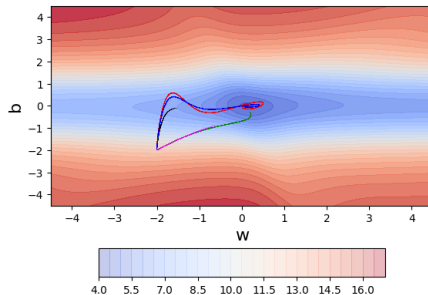
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



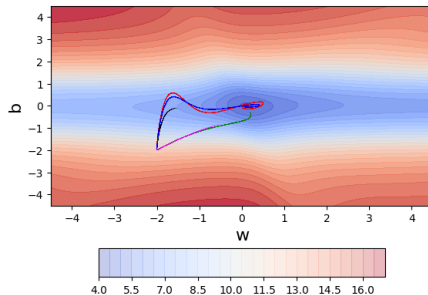
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



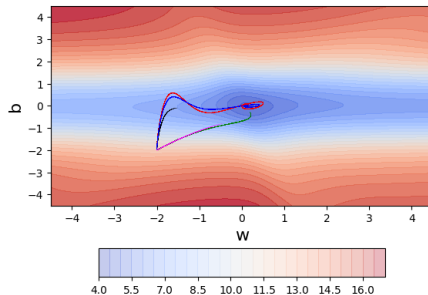
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



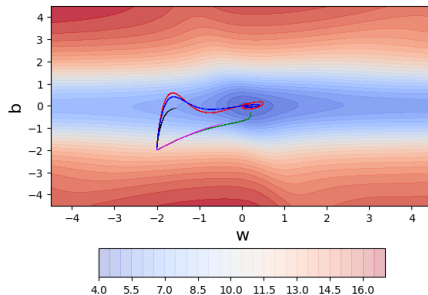
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



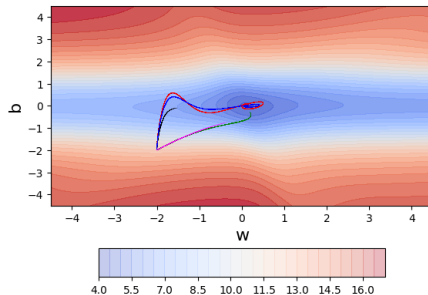
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



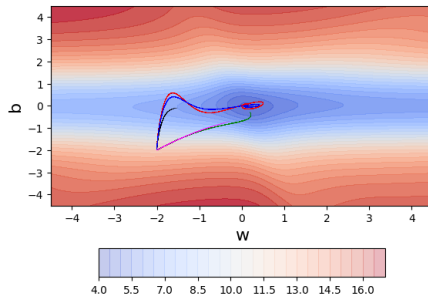

```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



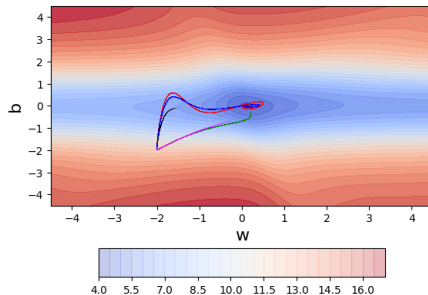
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



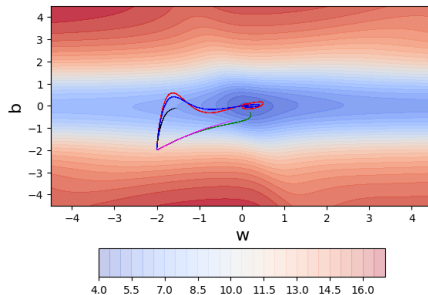
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



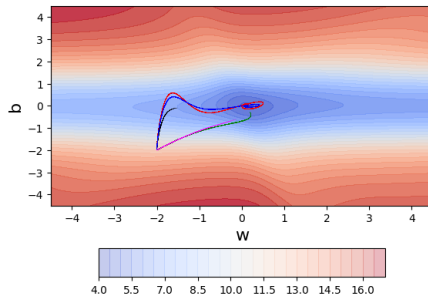
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



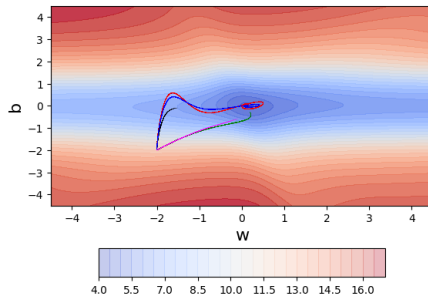
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

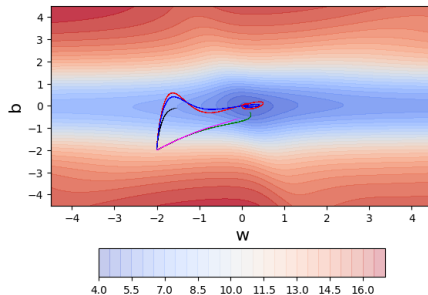
```



```
def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db
```



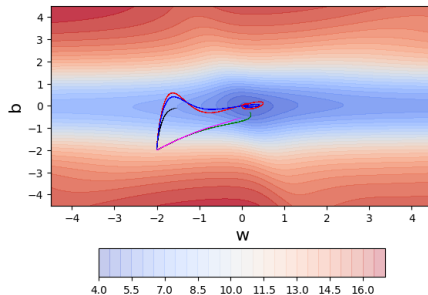
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



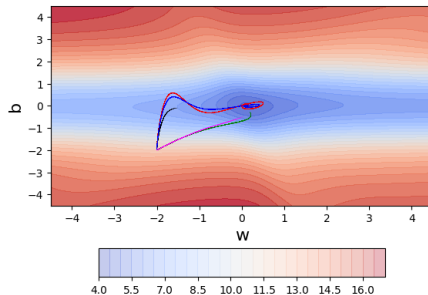
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



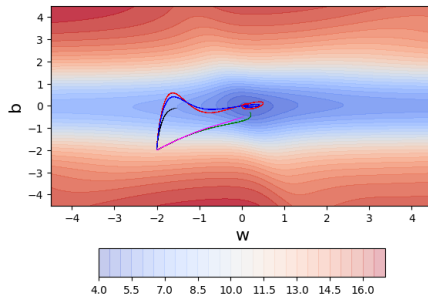

```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



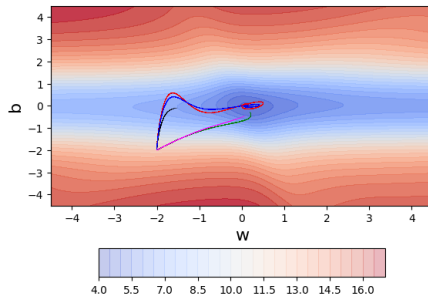
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



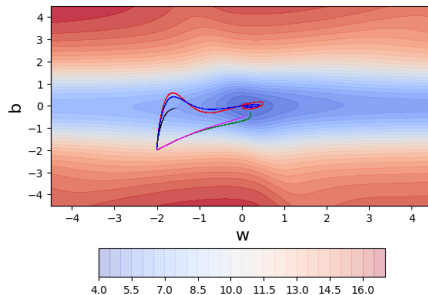
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



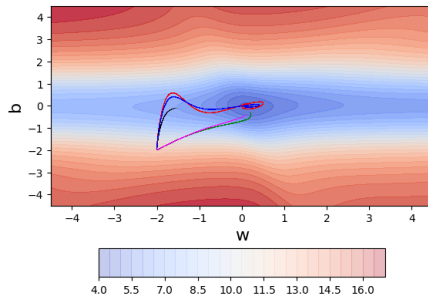
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



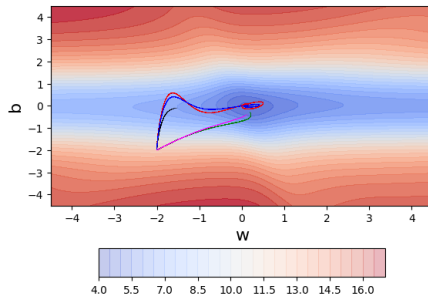
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



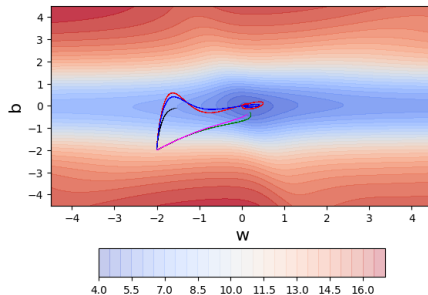
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



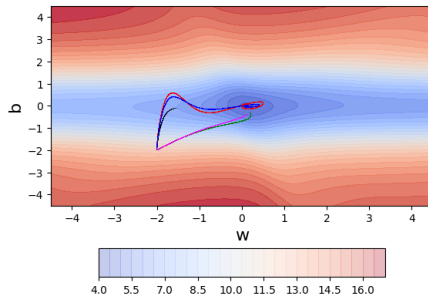
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



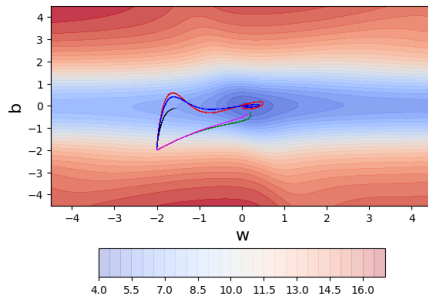
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



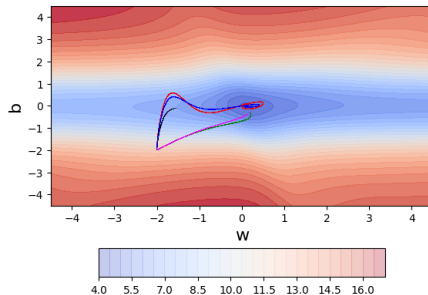

```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



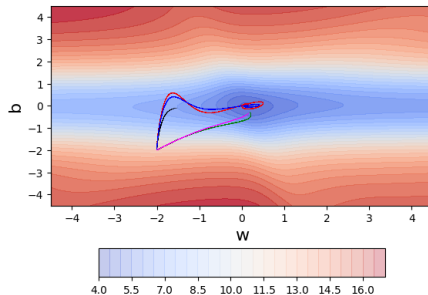
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



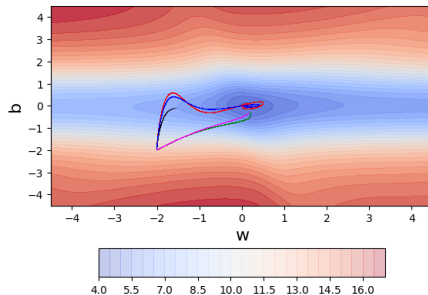
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



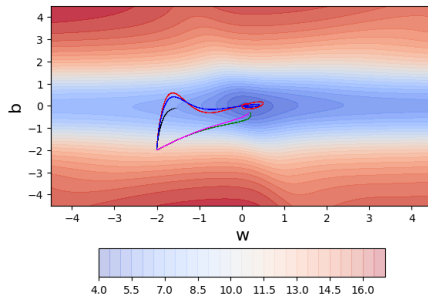
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



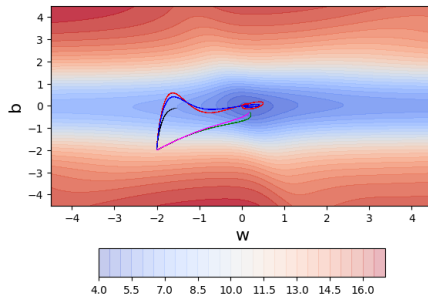
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

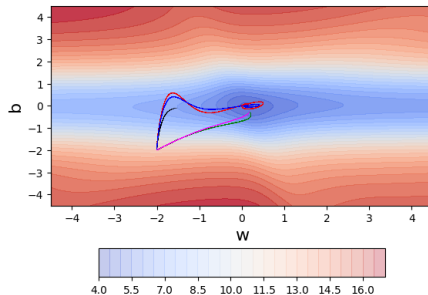
```



```
def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db
```



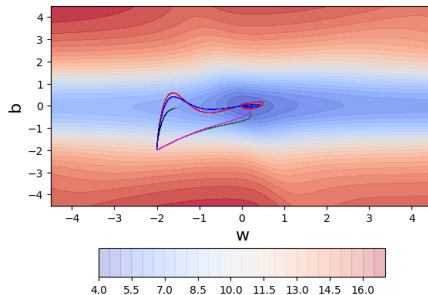
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

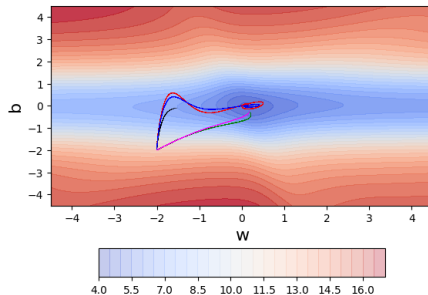
```



```
def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db
```



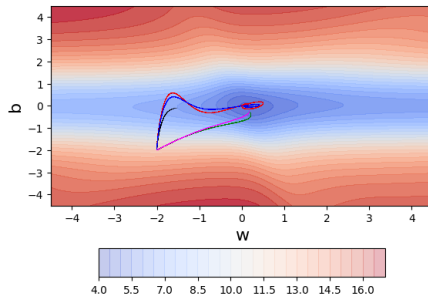

```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



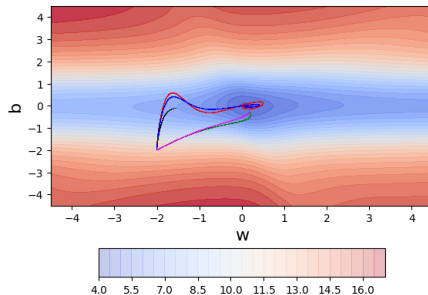
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



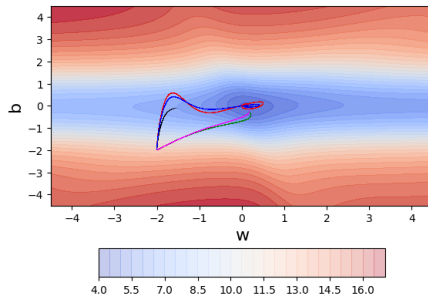
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



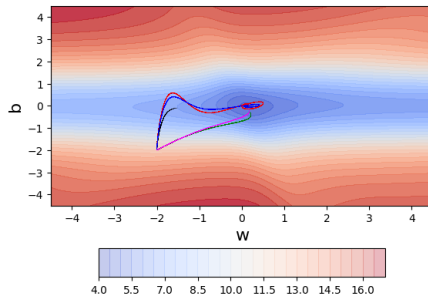
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

```



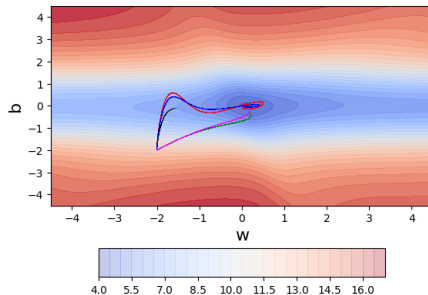
```

def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db

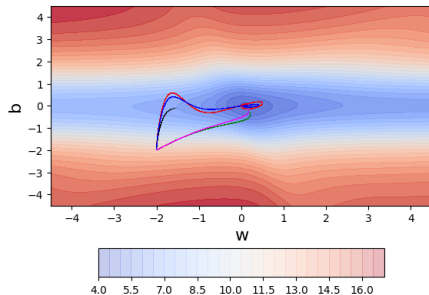
```



```
def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db
```

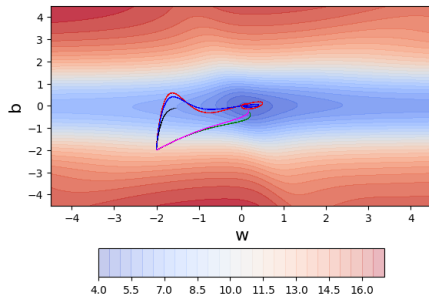


- Adagrad got stuck when it was close to convergence (it was no longer able to move in the vertical (b) direction because of the decayed learning rate)

```
def do_rmsprop() :
    w, b, eta = init_w, init_b, 0.1
    v_w, b_updates, eps, betal = 0, 0, 1e-8, 0.9
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        v_w = betal * v_w + (1 - betal) dw**2
        v_b = betal * v_b + (1 - betal) db**2

        w = w - (eta / np.sqrt(v_w + eps)) * dw
        b = b - (eta / np.sqrt(v_b + eps)) * db
```



- Adagrad got stuck when it was close to convergence (it was no longer able to move in the vertical (b) direction because of the decayed learning rate)

- RMSProp overcomes this problem by being less aggressive on the decay

Intuition

- Do everything that RMSProp does to solve the decay problem of Adagrad

Intuition

- Do everything that RMSProp does to solve the decay problem of Adagrad
- Plus use a cumulative history of the gradients

Intuition

- Do everything that RMSProp does to solve the decay problem of Adagrad
- Plus use a cumulative history of the gradients

Update rule for Adam

Intuition

- Do everything that RMSProp does to solve the decay problem of Adagrad
- Plus use a cumulative history of the gradients

Update rule for Adam

$$m_t = \beta_1 * m_{t-1} + (1 - \beta_1) * \nabla w_t$$

Intuition

- Do everything that RMSProp does to solve the decay problem of Adagrad
- Plus use a cumulative history of the gradients

Update rule for Adam

$$m_t = \beta_1 * m_{t-1} + (1 - \beta_1) * \nabla w_t$$
$$v_t = \beta_2 * v_{t-1} + (1 - \beta_2) * (\nabla w_t)^2$$

Intuition

- Do everything that RMSProp does to solve the decay problem of Adagrad
- Plus use a cumulative history of the gradients

Update rule for Adam

$$\begin{aligned}m_t &= \beta_1 * m_{t-1} + (1 - \beta_1) * \nabla w_t \\v_t &= \beta_2 * v_{t-1} + (1 - \beta_2) * (\nabla w_t)^2 \\ \hat{m}_t &= \frac{m_t}{1 - \beta_1^t}\end{aligned}$$

Intuition

- Do everything that RMSProp does to solve the decay problem of Adagrad
- Plus use a cumulative history of the gradients
- In practice, $\beta_1 = 0.9$ and $\beta_2 = 0.999$

Update rule for Adam

$$\begin{aligned}m_t &= \beta_1 * m_{t-1} + (1 - \beta_1) * \nabla w_t \\v_t &= \beta_2 * v_{t-1} + (1 - \beta_2) * (\nabla w_t)^2 \\ \hat{m}_t &= \frac{m_t}{1 - \beta_1^t} \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}\end{aligned}$$

Intuition

- Do everything that RMSProp does to solve the decay problem of Adagrad
- Plus use a cumulative history of the gradients
- In practice, $\beta_1 = 0.9$ and $\beta_2 = 0.999$

Update rule for Adam

$$m_t = \beta_1 * m_{t-1} + (1 - \beta_1) * \nabla w_t$$

$$v_t = \beta_2 * v_{t-1} + (1 - \beta_2) * (\nabla w_t)^2$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} * \hat{m}_t$$

Intuition

- Do everything that RMSProp does to solve the decay problem of Adagrad
- Plus use a cumulative history of the gradients
- In practice, $\beta_1 = 0.9$ and $\beta_2 = 0.999$

Update rule for Adam

$$m_t = \beta_1 * m_{t-1} + (1 - \beta_1) * \nabla w_t$$

$$v_t = \beta_2 * v_{t-1} + (1 - \beta_2) * (\nabla w_t)^2$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} * \hat{m}_t$$

... and a similar set of equations for b_t


```

def do_adam() :
    w_b_dw_db = [(init_w, init_b, 0, 0)]
    w_history, b_history, error_history = [], [], [
        ], []

    w, b, eta, mini_batch_size, num_points_seen =
        init_w, init_b, 0.1, 10, 0
    m_w, m_b, v_w, v_b, m_w_hat, m_b_hat, v_w_hat,
        v_b_hat, eps, beta1, beta2 = 0, 0, 0, 0, 0,
        0, 0, 0, 1e-8, 0.9, 0.999
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        m_w = beta1 * m_w + (1-beta1)*dw
        m_b = beta1 * m_b + (1-beta1)*db

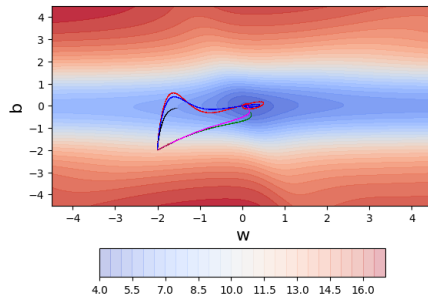
        v_w = beta2 * v_w + (1-beta2)*dw**2
        v_b = beta2 * v_b + (1-beta2)*db**2

        m_w_hat = m_w/(1-math.pow(beta1,i+1))
        m_b_hat = m_b/(1-math.pow(beta1,i+1))

        v_w_hat = v_w/(1-math.pow(beta2,i+1))
        v_b_hat = v_b/(1-math.pow(beta2,i+1))

        w = w - (eta / np.sqrt(v_w_hat + eps)) *
            m_w_hat
        b = b - (eta / np.sqrt(v_b_hat + eps)) *
            m_b_hat

```



```

def do_adam() :
    w_b_dw_db = [(init_w, init_b, 0, 0)]
    w_history, b_history, error_history = [], [], [
        ], []

    w, b, eta, mini_batch_size, num_points_seen =
        init_w, init_b, 0.1, 10, 0
    m_w, m_b, v_w, v_b, m_w_hat, m_b_hat, v_w_hat,
        v_b_hat, eps, beta1, beta2 = 0, 0, 0, 0, 0,
        0, 0, 0, 1e-8, 0.9, 0.999
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        m_w = beta1 * m_w + (1-beta1)*dw
        m_b = beta1 * m_b + (1-beta1)*db

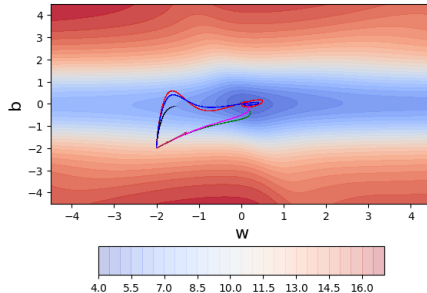
        v_w = beta2 * v_w + (1-beta2)*dw**2
        v_b = beta2 * v_b + (1-beta2)*db**2

        m_w_hat = m_w/(1-math.pow(beta1,i+1))
        m_b_hat = m_b/(1-math.pow(beta1,i+1))

        v_w_hat = v_w/(1-math.pow(beta2,i+1))
        v_b_hat = v_b/(1-math.pow(beta2,i+1))

        w = w - (eta / np.sqrt(v_w_hat + eps)) *
            m_w_hat
        b = b - (eta / np.sqrt(v_b_hat + eps)) *
            m_b_hat

```



```

def do_adam() :
    w_b_dw_db = [(init_w, init_b, 0, 0)]
    w_history, b_history, error_history = [], [], [
        ], []

    w, b, eta, mini_batch_size, num_points_seen =
        init_w, init_b, 0.1, 10, 0
    m_w, m_b, v_w, v_b, m_w_hat, m_b_hat, v_w_hat,
        v_b_hat, eps, beta1, beta2 = 0, 0, 0, 0, 0,
        0, 0, 0, 1e-8, 0.9, 0.999
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        m_w = beta1 * m_w + (1-beta1)*dw
        m_b = beta1 * m_b + (1-beta1)*db

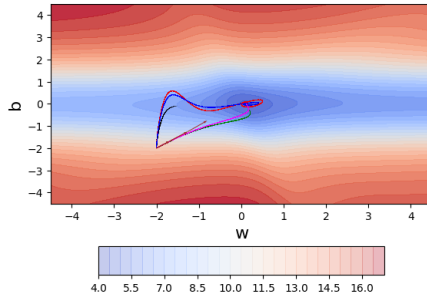
        v_w = beta2 * v_w + (1-beta2)*dw**2
        v_b = beta2 * v_b + (1-beta2)*db**2

        m_w_hat = m_w/(1-math.pow(beta1,i+1))
        m_b_hat = m_b/(1-math.pow(beta1,i+1))

        v_w_hat = v_w/(1-math.pow(beta2,i+1))
        v_b_hat = v_b/(1-math.pow(beta2,i+1))

        w = w - (eta / np.sqrt(v_w_hat + eps)) *
            m_w_hat
        b = b - (eta / np.sqrt(v_b_hat + eps)) *
            m_b_hat

```



```

def do_adam() :
    w_b_dw_db = [(init_w, init_b, 0, 0)]
    w_history, b_history, error_history = [], [], [
        ], []

    w, b, eta, mini_batch_size, num_points_seen =
        init_w, init_b, 0.1, 10, 0
    m_w, m_b, v_w, v_b, m_w_hat, m_b_hat, v_w_hat,
        v_b_hat, eps, beta1, beta2 = 0, 0, 0, 0, 0,
        0, 0, 0, 1e-8, 0.9, 0.999
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        m_w = beta1 * m_w + (1-beta1)*dw
        m_b = beta1 * m_b + (1-beta1)*db

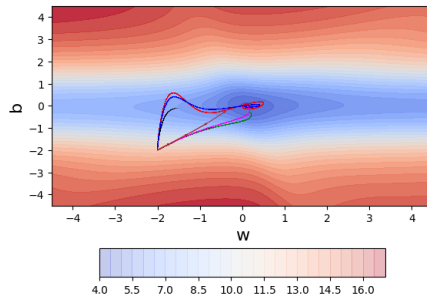
        v_w = beta2 * v_w + (1-beta2)*dw**2
        v_b = beta2 * v_b + (1-beta2)*db**2

        m_w_hat = m_w/(1-math.pow(beta1,i+1))
        m_b_hat = m_b/(1-math.pow(beta1,i+1))

        v_w_hat = v_w/(1-math.pow(beta2,i+1))
        v_b_hat = v_b/(1-math.pow(beta2,i+1))

        w = w - (eta / np.sqrt(v_w_hat + eps)) *
            m_w_hat
        b = b - (eta / np.sqrt(v_b_hat + eps)) *
            m_b_hat

```



```

def do_adam() :
    w_b_dw_db = [(init_w, init_b, 0, 0)]
    w_history, b_history, error_history = [], [], [
        ], []

    w, b, eta, mini_batch_size, num_points_seen =
        init_w, init_b, 0.1, 10, 0
    m_w, m_b, v_w, v_b, m_w_hat, m_b_hat, v_w_hat,
        v_b_hat, eps, beta1, beta2 = 0, 0, 0, 0, 0,
        0, 0, 0, 1e-8, 0.9, 0.999
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        m_w = beta1 * m_w + (1-beta1)*dw
        m_b = beta1 * m_b + (1-beta1)*db

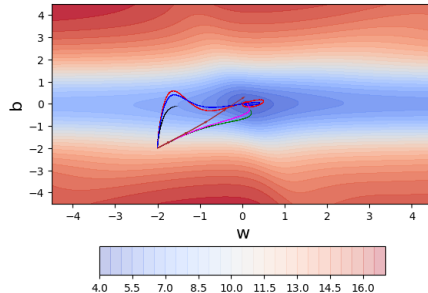
        v_w = beta2 * v_w + (1-beta2)*dw**2
        v_b = beta2 * v_b + (1-beta2)*db**2

        m_w_hat = m_w/(1-math.pow(beta1,i+1))
        m_b_hat = m_b/(1-math.pow(beta1,i+1))

        v_w_hat = v_w/(1-math.pow(beta2,i+1))
        v_b_hat = v_b/(1-math.pow(beta2,i+1))

        w = w - (eta / np.sqrt(v_w_hat + eps)) *
            m_w_hat
        b = b - (eta / np.sqrt(v_b_hat + eps)) *
            m_b_hat

```



```

def do_adam() :
    w_b_dw_db = [(init_w, init_b, 0, 0)]
    w_history, b_history, error_history = [], [], [
        ], []

    w, b, eta, mini_batch_size, num_points_seen =
        init_w, init_b, 0.1, 10, 0
    m_w, m_b, v_w, v_b, m_w_hat, m_b_hat, v_w_hat,
        v_b_hat, eps, beta1, beta2 = 0, 0, 0, 0, 0,
        0, 0, 0, 1e-8, 0.9, 0.999
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        m_w = beta1 * m_w + (1-beta1)*dw
        m_b = beta1 * m_b + (1-beta1)*db

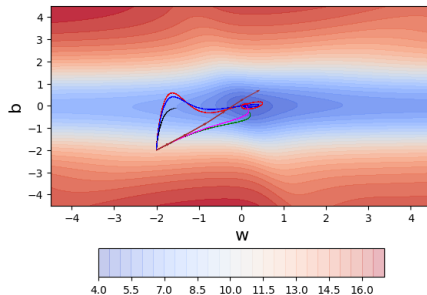
        v_w = beta2 * v_w + (1-beta2)*dw**2
        v_b = beta2 * v_b + (1-beta2)*db**2

        m_w_hat = m_w/(1-math.pow(beta1,i+1))
        m_b_hat = m_b/(1-math.pow(beta1,i+1))

        v_w_hat = v_w/(1-math.pow(beta2,i+1))
        v_b_hat = v_b/(1-math.pow(beta2,i+1))

        w = w - (eta / np.sqrt(v_w_hat + eps)) *
            m_w_hat
        b = b - (eta / np.sqrt(v_b_hat + eps)) *
            m_b_hat

```



```

def do_adam() :
    w_b_dw_db = [(init_w, init_b, 0, 0)]
    w_history, b_history, error_history = [], [], [
        ], []

    w, b, eta, mini_batch_size, num_points_seen =
        init_w, init_b, 0.1, 10, 0
    m_w, m_b, v_w, v_b, m_w_hat, m_b_hat, v_w_hat,
        v_b_hat, eps, beta1, beta2 = 0, 0, 0, 0, 0,
        0, 0, 0, 1e-8, 0.9, 0.999
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        m_w = beta1 * m_w + (1-beta1)*dw
        m_b = beta1 * m_b + (1-beta1)*db

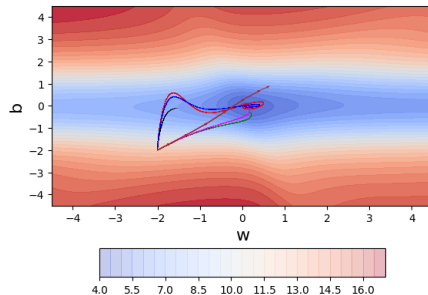
        v_w = beta2 * v_w + (1-beta2)*dw**2
        v_b = beta2 * v_b + (1-beta2)*db**2

        m_w_hat = m_w/(1-math.pow(beta1,i+1))
        m_b_hat = m_b/(1-math.pow(beta1,i+1))

        v_w_hat = v_w/(1-math.pow(beta2,i+1))
        v_b_hat = v_b/(1-math.pow(beta2,i+1))

        w = w - (eta / np.sqrt(v_w_hat + eps)) *
            m_w_hat
        b = b - (eta / np.sqrt(v_b_hat + eps)) *
            m_b_hat

```



```

def do_adam() :
    w_b_dw_db = [(init_w, init_b, 0, 0)]
    w_history, b_history, error_history = [], [], [
        ], []

    w, b, eta, mini_batch_size, num_points_seen =
        init_w, init_b, 0.1, 10, 0
    m_w, m_b, v_w, v_b, m_w_hat, m_b_hat, v_w_hat,
        v_b_hat, eps, beta1, beta2 = 0, 0, 0, 0, 0,
        0, 0, 0, 1e-8, 0.9, 0.999
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        m_w = beta1 * m_w + (1-beta1)*dw
        m_b = beta1 * m_b + (1-beta1)*db

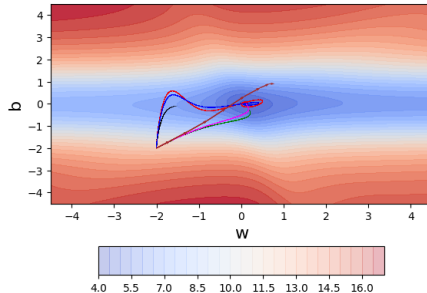
        v_w = beta2 * v_w + (1-beta2)*dw**2
        v_b = beta2 * v_b + (1-beta2)*db**2

        m_w_hat = m_w/(1-math.pow(beta1,i+1))
        m_b_hat = m_b/(1-math.pow(beta1,i+1))

        v_w_hat = v_w/(1-math.pow(beta2,i+1))
        v_b_hat = v_b/(1-math.pow(beta2,i+1))

        w = w - (eta / np.sqrt(v_w_hat + eps)) *
            m_w_hat
        b = b - (eta / np.sqrt(v_b_hat + eps)) *
            m_b_hat

```




```

def do_adam() :
    w_b_dw_db = [(init_w, init_b, 0, 0)]
    w_history, b_history, error_history = [], [], []

    w, b, eta, mini_batch_size, num_points_seen =
        init_w, init_b, 0.1, 10, 0
    m_w, m_b, v_w, v_b, m_w_hat, m_b_hat, v_w_hat,
        v_b_hat, eps, beta1, beta2 = 0, 0, 0, 0, 0,
        0, 0, 0, 1e-8, 0.9, 0.999
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        m_w = beta1 * m_w + (1-beta1)*dw
        m_b = beta1 * m_b + (1-beta1)*db

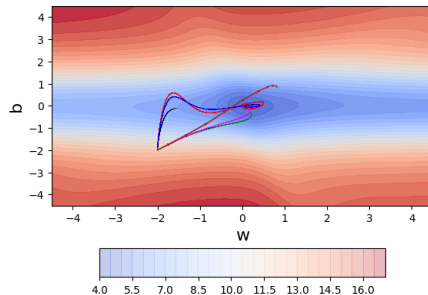
        v_w = beta2 * v_w + (1-beta2)*dw**2
        v_b = beta2 * v_b + (1-beta2)*db**2

        m_w_hat = m_w/(1-math.pow(beta1,i+1))
        m_b_hat = m_b/(1-math.pow(beta1,i+1))

        v_w_hat = v_w/(1-math.pow(beta2,i+1))
        v_b_hat = v_b/(1-math.pow(beta2,i+1))

        w = w - (eta / np.sqrt(v_w_hat + eps)) *
            m_w_hat
        b = b - (eta / np.sqrt(v_b_hat + eps)) *
            m_b_hat

```



```

def do_adam() :
    w_b_dw_db = [(init_w, init_b, 0, 0)]
    w_history, b_history, error_history = [], [], [
        ], []

    w, b, eta, mini_batch_size, num_points_seen =
        init_w, init_b, 0.1, 10, 0
    m_w, m_b, v_w, v_b, m_w_hat, m_b_hat, v_w_hat,
        v_b_hat, eps, beta1, beta2 = 0, 0, 0, 0, 0,
        0, 0, 0, 1e-8, 0.9, 0.999
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        m_w = beta1 * m_w + (1-beta1)*dw
        m_b = beta1 * m_b + (1-beta1)*db

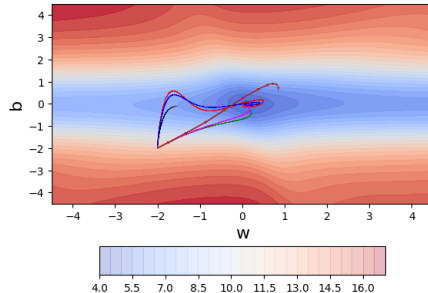
        v_w = beta2 * v_w + (1-beta2)*dw**2
        v_b = beta2 * v_b + (1-beta2)*db**2

        m_w_hat = m_w/(1-math.pow(beta1,i+1))
        m_b_hat = m_b/(1-math.pow(beta1,i+1))

        v_w_hat = v_w/(1-math.pow(beta2,i+1))
        v_b_hat = v_b/(1-math.pow(beta2,i+1))

        w = w - (eta / np.sqrt(v_w_hat + eps)) *
            m_w_hat
        b = b - (eta / np.sqrt(v_b_hat + eps)) *
            m_b_hat

```



```

def do_adam() :
    w_b_dw_db = [(init_w, init_b, 0, 0)]
    w_history, b_history, error_history = [], [], [
        ], []

    w, b, eta, mini_batch_size, num_points_seen =
        init_w, init_b, 0.1, 10, 0
    m_w, m_b, v_w, v_b, m_w_hat, m_b_hat, v_w_hat,
        v_b_hat, eps, beta1, beta2 = 0, 0, 0, 0, 0,
        0, 0, 0, 1e-8, 0.9, 0.999
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        m_w = beta1 * m_w + (1-beta1)*dw
        m_b = beta1 * m_b + (1-beta1)*db

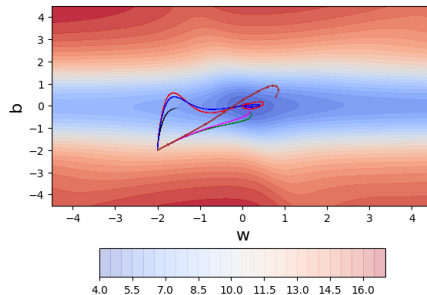
        v_w = beta2 * v_w + (1-beta2)*dw**2
        v_b = beta2 * v_b + (1-beta2)*db**2

        m_w_hat = m_w/(1-math.pow(beta1,i+1))
        m_b_hat = m_b/(1-math.pow(beta1,i+1))

        v_w_hat = v_w/(1-math.pow(beta2,i+1))
        v_b_hat = v_b/(1-math.pow(beta2,i+1))

        w = w - (eta / np.sqrt(v_w_hat + eps)) *
            m_w_hat
        b = b - (eta / np.sqrt(v_b_hat + eps)) *
            m_b_hat

```



```

def do_adam() :
    w_b_dw_db = [(init_w, init_b, 0, 0)]
    w_history, b_history, error_history = [], [], [
        ], [
    ], [

    w, b, eta, mini_batch_size, num_points_seen =
        init_w, init_b, 0.1, 10, 0
    m_w, m_b, v_w, v_b, m_w_hat, m_b_hat, v_w_hat,
        v_b_hat, eps, beta1, beta2 = 0, 0, 0, 0, 0,
        0, 0, 0, 1e-8, 0.9, 0.999
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        m_w = beta1 * m_w + (1-beta1)*dw
        m_b = beta1 * m_b + (1-beta1)*db

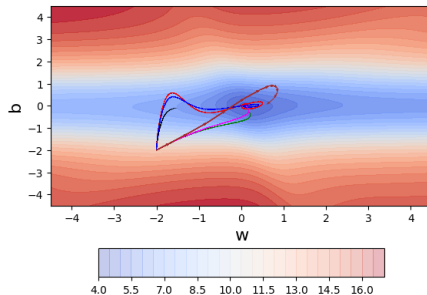
        v_w = beta2 * v_w + (1-beta2)*dw**2
        v_b = beta2 * v_b + (1-beta2)*db**2

        m_w_hat = m_w/(1-math.pow(beta1,i+1))
        m_b_hat = m_b/(1-math.pow(beta1,i+1))

        v_w_hat = v_w/(1-math.pow(beta2,i+1))
        v_b_hat = v_b/(1-math.pow(beta2,i+1))

        w = w - (eta / np.sqrt(v_w_hat + eps)) *
            m_w_hat
        b = b - (eta / np.sqrt(v_b_hat + eps)) *
            m_b_hat

```



```

def do_adam() :
    w_b_dw_db = [(init_w, init_b, 0, 0)]
    w_history, b_history, error_history = [], [], [
        ], []

    w, b, eta, mini_batch_size, num_points_seen =
        init_w, init_b, 0.1, 10, 0
    m_w, m_b, v_w, v_b, m_w_hat, m_b_hat, v_w_hat,
        v_b_hat, eps, beta1, beta2 = 0, 0, 0, 0, 0,
        0, 0, 0, 1e-8, 0.9, 0.999
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        m_w = beta1 * m_w + (1-beta1)*dw
        m_b = beta1 * m_b + (1-beta1)*db

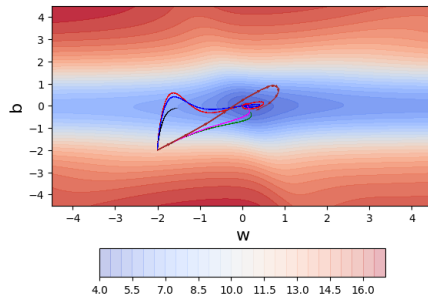
        v_w = beta2 * v_w + (1-beta2)*dw**2
        v_b = beta2 * v_b + (1-beta2)*db**2

        m_w_hat = m_w/(1-math.pow(beta1,i+1))
        m_b_hat = m_b/(1-math.pow(beta1,i+1))

        v_w_hat = v_w/(1-math.pow(beta2,i+1))
        v_b_hat = v_b/(1-math.pow(beta2,i+1))

        w = w - (eta / np.sqrt(v_w_hat + eps)) *
            m_w_hat
        b = b - (eta / np.sqrt(v_b_hat + eps)) *
            m_b_hat

```



```

def do_adam() :
    w_b_dw_db = [(init_w, init_b, 0, 0)]
    w_history, b_history, error_history = [], [], [
        ], [
    ], [

    w, b, eta, mini_batch_size, num_points_seen =
        init_w, init_b, 0.1, 10, 0
    m_w, m_b, v_w, v_b, m_w_hat, m_b_hat, v_w_hat,
        v_b_hat, eps, beta1, beta2 = 0, 0, 0, 0, 0,
        0, 0, 0, 1e-8, 0.9, 0.999
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        m_w = beta1 * m_w + (1-beta1)*dw
        m_b = beta1 * m_b + (1-beta1)*db

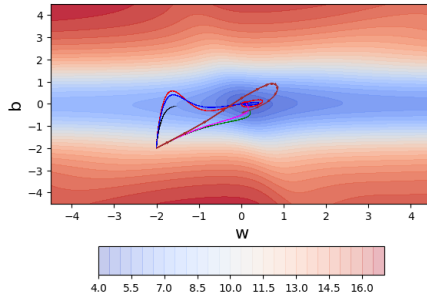
        v_w = beta2 * v_w + (1-beta2)*dw**2
        v_b = beta2 * v_b + (1-beta2)*db**2

        m_w_hat = m_w/(1-math.pow(beta1,i+1))
        m_b_hat = m_b/(1-math.pow(beta1,i+1))

        v_w_hat = v_w/(1-math.pow(beta2,i+1))
        v_b_hat = v_b/(1-math.pow(beta2,i+1))

        w = w - (eta / np.sqrt(v_w_hat + eps)) *
            m_w_hat
        b = b - (eta / np.sqrt(v_b_hat + eps)) *
            m_b_hat

```



```

def do_adam() :
    w_b_dw_db = [(init_w, init_b, 0, 0)]
    w_history, b_history, error_history = [], [], [
    ], []

    w, b, eta, mini_batch_size, num_points_seen =
        init_w, init_b, 0.1, 10, 0
    m_w, m_b, v_w, v_b, m_w_hat, m_b_hat, v_w_hat,
        v_b_hat, eps, beta1, beta2 = 0, 0, 0, 0, 0,
        0, 0, 0, 1e-8, 0.9, 0.999
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        m_w = beta1 * m_w + (1-beta1)*dw
        m_b = beta1 * m_b + (1-beta1)*db

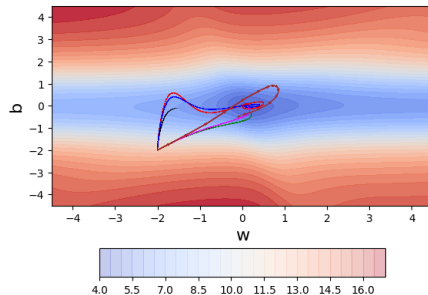
        v_w = beta2 * v_w + (1-beta2)*dw**2
        v_b = beta2 * v_b + (1-beta2)*db**2

        m_w_hat = m_w/(1-math.pow(beta1,i+1))
        m_b_hat = m_b/(1-math.pow(beta1,i+1))

        v_w_hat = v_w/(1-math.pow(beta2,i+1))
        v_b_hat = v_b/(1-math.pow(beta2,i+1))

        w = w - (eta / np.sqrt(v_w_hat + eps)) *
            m_w_hat
        b = b - (eta / np.sqrt(v_b_hat + eps)) *
            m_b_hat

```



```

def do_adam() :
    w_b_dw_db = [(init_w, init_b, 0, 0)]
    w_history, b_history, error_history = [], [], []

    w, b, eta, mini_batch_size, num_points_seen =
        init_w, init_b, 0.1, 10, 0
    m_w, m_b, v_w, v_b, m_w_hat, m_b_hat, v_w_hat,
        v_b_hat, eps, beta1, beta2 = 0, 0, 0, 0, 0,
        0, 0, 0, 1e-8, 0.9, 0.999
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        m_w = beta1 * m_w + (1-beta1)*dw
        m_b = beta1 * m_b + (1-beta1)*db

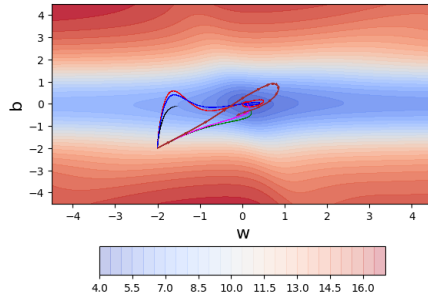
        v_w = beta2 * v_w + (1-beta2)*dw**2
        v_b = beta2 * v_b + (1-beta2)*db**2

        m_w_hat = m_w/(1-math.pow(beta1,i+1))
        m_b_hat = m_b/(1-math.pow(beta1,i+1))

        v_w_hat = v_w/(1-math.pow(beta2,i+1))
        v_b_hat = v_b/(1-math.pow(beta2,i+1))

        w = w - (eta / np.sqrt(v_w_hat + eps)) *
            m_w_hat
        b = b - (eta / np.sqrt(v_b_hat + eps)) *
            m_b_hat

```




```

def do_adam() :
    w_b_dw_db = [(init_w, init_b, 0, 0)]
    w_history, b_history, error_history = [], [], [
    ], []

    w, b, eta, mini_batch_size, num_points_seen =
        init_w, init_b, 0.1, 10, 0
    m_w, m_b, v_w, v_b, m_w_hat, m_b_hat, v_w_hat,
        v_b_hat, eps, beta1, beta2 = 0, 0, 0, 0, 0
        , 0, 0, 0, 1e-8, 0.9, 0.999
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        m_w = beta1 * m_w + (1-beta1)*dw
        m_b = beta1 * m_b + (1-beta1)*db

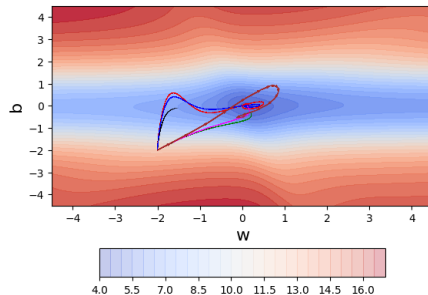
        v_w = beta2 * v_w + (1-beta2)*dw**2
        v_b = beta2 * v_b + (1-beta2)*db**2

        m_w_hat = m_w/(1-math.pow(beta1,i+1))
        m_b_hat = m_b/(1-math.pow(beta1,i+1))

        v_w_hat = v_w/(1-math.pow(beta2,i+1))
        v_b_hat = v_b/(1-math.pow(beta2,i+1))

        w = w - (eta / np.sqrt(v_w_hat + eps)) *
            m_w_hat
        b = b - (eta / np.sqrt(v_b_hat + eps)) *
            m_b_hat

```



```

def do_adam() :
    w_b_dw_db = [(init_w, init_b, 0, 0)]
    w_history, b_history, error_history = [], [], [
    ], []

    w, b, eta, mini_batch_size, num_points_seen =
        init_w, init_b, 0.1, 10, 0
    m_w, m_b, v_w, v_b, m_w_hat, m_b_hat, v_w_hat,
        v_b_hat, eps, beta1, beta2 = 0, 0, 0, 0, 0,
        0, 0, 0, 1e-8, 0.9, 0.999
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        m_w = beta1 * m_w + (1-beta1)*dw
        m_b = beta1 * m_b + (1-beta1)*db

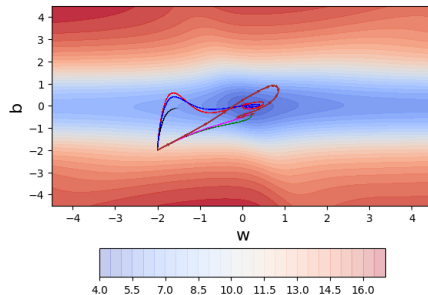
        v_w = beta2 * v_w + (1-beta2)*dw**2
        v_b = beta2 * v_b + (1-beta2)*db**2

        m_w_hat = m_w/(1-math.pow(beta1,i+1))
        m_b_hat = m_b/(1-math.pow(beta1,i+1))

        v_w_hat = v_w/(1-math.pow(beta2,i+1))
        v_b_hat = v_b/(1-math.pow(beta2,i+1))

        w = w - (eta / np.sqrt(v_w_hat + eps)) *
            m_w_hat
        b = b - (eta / np.sqrt(v_b_hat + eps)) *
            m_b_hat

```



```

def do_adam() :
    w_b_dw_db = [(init_w, init_b, 0, 0)]
    w_history, b_history, error_history = [], [], []

    w, b, eta, mini_batch_size, num_points_seen =
        init_w, init_b, 0.1, 10, 0
    m_w, m_b, v_w, v_b, m_w_hat, m_b_hat, v_w_hat,
        v_b_hat, eps, beta1, beta2 = 0, 0, 0, 0, 0,
        0, 0, 0, 1e-8, 0.9, 0.999
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        m_w = beta1 * m_w + (1-beta1)*dw
        m_b = beta1 * m_b + (1-beta1)*db

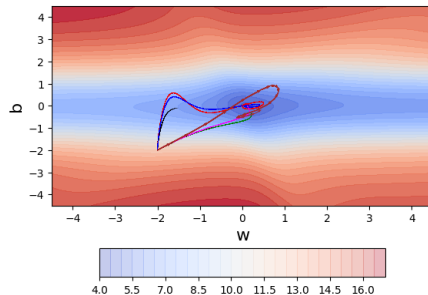
        v_w = beta2 * v_w + (1-beta2)*dw**2
        v_b = beta2 * v_b + (1-beta2)*db**2

        m_w_hat = m_w/(1-math.pow(beta1,i+1))
        m_b_hat = m_b/(1-math.pow(beta1,i+1))

        v_w_hat = v_w/(1-math.pow(beta2,i+1))
        v_b_hat = v_b/(1-math.pow(beta2,i+1))

        w = w - (eta / np.sqrt(v_w_hat + eps)) *
            m_w_hat
        b = b - (eta / np.sqrt(v_b_hat + eps)) *
            m_b_hat

```



```

def do_adam() :
    w_b_dw_db = [(init_w, init_b, 0, 0)]
    w_history, b_history, error_history = [], [], []

    w, b, eta, mini_batch_size, num_points_seen =
        init_w, init_b, 0.1, 10, 0
    m_w, m_b, v_w, v_b, m_w_hat, m_b_hat, v_w_hat,
        v_b_hat, eps, beta1, beta2 = 0, 0, 0, 0, 0, 0, 0, 0, 0, 0.9, 0.999
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        m_w = beta1 * m_w + (1-beta1)*dw
        m_b = beta1 * m_b + (1-beta1)*db

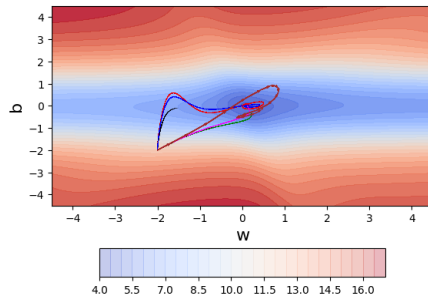
        v_w = beta2 * v_w + (1-beta2)*dw**2
        v_b = beta2 * v_b + (1-beta2)*db**2

        m_w_hat = m_w/(1-math.pow(beta1,i+1))
        m_b_hat = m_b/(1-math.pow(beta1,i+1))

        v_w_hat = v_w/(1-math.pow(beta2,i+1))
        v_b_hat = v_b/(1-math.pow(beta2,i+1))

        w = w - (eta / np.sqrt(v_w_hat + eps)) *
            m_w_hat
        b = b - (eta / np.sqrt(v_b_hat + eps)) *
            m_b_hat

```



```

def do_adam() :
    w_b_dw_db = [(init_w, init_b, 0, 0)]
    w_history, b_history, error_history = [], [], [
    ], []

    w, b, eta, mini_batch_size, num_points_seen =
        init_w, init_b, 0.1, 10, 0
    m_w, m_b, v_w, v_b, m_w_hat, m_b_hat, v_w_hat,
        v_b_hat, eps, beta1, beta2 = 0, 0, 0, 0, 0,
        0, 0, 0, 1e-8, 0.9, 0.999
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        m_w = beta1 * m_w + (1-beta1)*dw
        m_b = beta1 * m_b + (1-beta1)*db

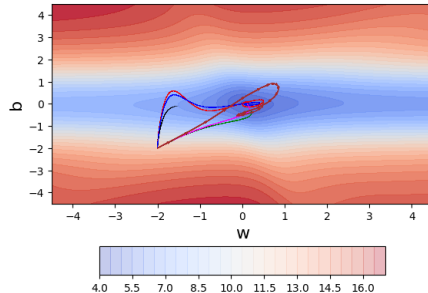
        v_w = beta2 * v_w + (1-beta2)*dw**2
        v_b = beta2 * v_b + (1-beta2)*db**2

        m_w_hat = m_w/(1-math.pow(beta1,i+1))
        m_b_hat = m_b/(1-math.pow(beta1,i+1))

        v_w_hat = v_w/(1-math.pow(beta2,i+1))
        v_b_hat = v_b/(1-math.pow(beta2,i+1))

        w = w - (eta / np.sqrt(v_w_hat + eps)) *
            m_w_hat
        b = b - (eta / np.sqrt(v_b_hat + eps)) *
            m_b_hat

```



```

def do_adam() :
    w_b_dw_db = [(init_w, init_b, 0, 0)]
    w_history, b_history, error_history = [], [], [
        ], []

    w, b, eta, mini_batch_size, num_points_seen =
        init_w, init_b, 0.1, 10, 0
    m_w, m_b, v_w, v_b, m_w_hat, m_b_hat, v_w_hat,
        v_b_hat, eps, beta1, beta2 = 0, 0, 0, 0, 0,
        0, 0, 0, 1e-8, 0.9, 0.999
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        m_w = beta1 * m_w + (1-beta1)*dw
        m_b = beta1 * m_b + (1-beta1)*db

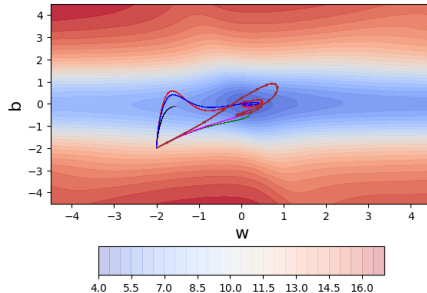
        v_w = beta2 * v_w + (1-beta2)*dw**2
        v_b = beta2 * v_b + (1-beta2)*db**2

        m_w_hat = m_w/(1-math.pow(beta1,i+1))
        m_b_hat = m_b/(1-math.pow(beta1,i+1))

        v_w_hat = v_w/(1-math.pow(beta2,i+1))
        v_b_hat = v_b/(1-math.pow(beta2,i+1))

        w = w - (eta / np.sqrt(v_w_hat + eps)) *
            m_w_hat
        b = b - (eta / np.sqrt(v_b_hat + eps)) *
            m_b_hat

```



```

def do_adam() :
    w_b_dw_db = [(init_w, init_b, 0, 0)]
    w_history, b_history, error_history = [], [], [
        ], []

    w, b, eta, mini_batch_size, num_points_seen =
        init_w, init_b, 0.1, 10, 0
    m_w, m_b, v_w, v_b, m_w_hat, m_b_hat, v_w_hat,
        v_b_hat, eps, beta1, beta2 = 0, 0, 0, 0, 0,
        0, 0, 0, 1e-8, 0.9, 0.999
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        m_w = beta1 * m_w + (1-beta1)*dw
        m_b = beta1 * m_b + (1-beta1)*db

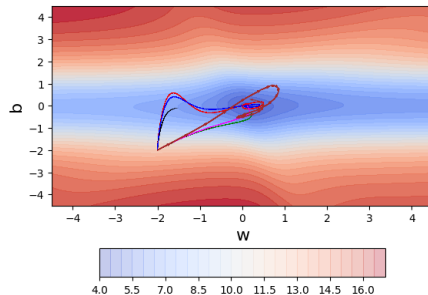
        v_w = beta2 * v_w + (1-beta2)*dw**2
        v_b = beta2 * v_b + (1-beta2)*db**2

        m_w_hat = m_w/(1-math.pow(beta1,i+1))
        m_b_hat = m_b/(1-math.pow(beta1,i+1))

        v_w_hat = v_w/(1-math.pow(beta2,i+1))
        v_b_hat = v_b/(1-math.pow(beta2,i+1))

        w = w - (eta / np.sqrt(v_w_hat + eps)) *
            m_w_hat
        b = b - (eta / np.sqrt(v_b_hat + eps)) *
            m_b_hat

```



```

def do_adam() :
    w_b_dw_db = [(init_w, init_b, 0, 0)]
    w_history, b_history, error_history = [], [], [
        ], []

    w, b, eta, mini_batch_size, num_points_seen =
        init_w, init_b, 0.1, 10, 0
    m_w, m_b, v_w, v_b, m_w_hat, m_b_hat, v_w_hat,
        v_b_hat, eps, beta1, beta2 = 0, 0, 0, 0, 0,
        0, 0, 0, 1e-8, 0.9, 0.999
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        m_w = beta1 * m_w + (1-beta1)*dw
        m_b = beta1 * m_b + (1-beta1)*db

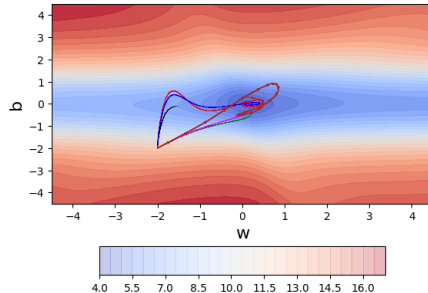
        v_w = beta2 * v_w + (1-beta2)*dw**2
        v_b = beta2 * v_b + (1-beta2)*db**2

        m_w_hat = m_w/(1-math.pow(beta1,i+1))
        m_b_hat = m_b/(1-math.pow(beta1,i+1))

        v_w_hat = v_w/(1-math.pow(beta2,i+1))
        v_b_hat = v_b/(1-math.pow(beta2,i+1))

        w = w - (eta / np.sqrt(v_w_hat + eps)) *
            m_w_hat
        b = b - (eta / np.sqrt(v_b_hat + eps)) *
            m_b_hat

```




```

def do_adam() :
    w_b_dw_db = [(init_w, init_b, 0, 0)]
    w_history, b_history, error_history = [], [], []

    w, b, eta, mini_batch_size, num_points_seen =
        init_w, init_b, 0.1, 10, 0
    m_w, m_b, v_w, v_b, m_w_hat, m_b_hat, v_w_hat,
        v_b_hat, eps, beta1, beta2 = 0, 0, 0, 0, 0,
        0, 0, 0, 1e-8, 0.9, 0.999
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        m_w = beta1 * m_w + (1-beta1)*dw
        m_b = beta1 * m_b + (1-beta1)*db

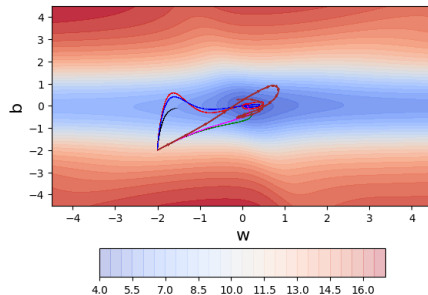
        v_w = beta2 * v_w + (1-beta2)*dw**2
        v_b = beta2 * v_b + (1-beta2)*db**2

        m_w_hat = m_w/(1-math.pow(beta1,i+1))
        m_b_hat = m_b/(1-math.pow(beta1,i+1))

        v_w_hat = v_w/(1-math.pow(beta2,i+1))
        v_b_hat = v_b/(1-math.pow(beta2,i+1))

        w = w - (eta / np.sqrt(v_w_hat + eps)) *
            m_w_hat
        b = b - (eta / np.sqrt(v_b_hat + eps)) *
            m_b_hat

```



```

def do_adam() :
    w_b_dw_db = [(init_w, init_b, 0, 0)]
    w_history, b_history, error_history = [], [], [
        ], [
    ], [

    w, b, eta, mini_batch_size, num_points_seen =
        init_w, init_b, 0.1, 10, 0
    m_w, m_b, v_w, v_b, m_w_hat, m_b_hat, v_w_hat,
        v_b_hat, eps, beta1, beta2 = 0, 0, 0, 0, 0,
        0, 0, 0, 1e-8, 0.9, 0.999
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        m_w = beta1 * m_w + (1-beta1)*dw
        m_b = beta1 * m_b + (1-beta1)*db

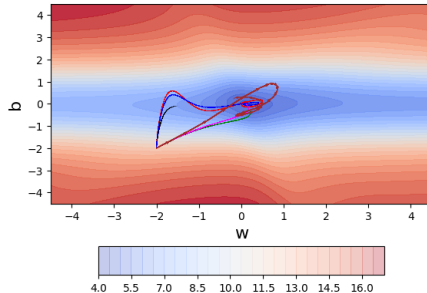
        v_w = beta2 * v_w + (1-beta2)*dw**2
        v_b = beta2 * v_b + (1-beta2)*db**2

        m_w_hat = m_w/(1-math.pow(beta1,i+1))
        m_b_hat = m_b/(1-math.pow(beta1,i+1))

        v_w_hat = v_w/(1-math.pow(beta2,i+1))
        v_b_hat = v_b/(1-math.pow(beta2,i+1))

        w = w - (eta / np.sqrt(v_w_hat + eps)) *
            m_w_hat
        b = b - (eta / np.sqrt(v_b_hat + eps)) *
            m_b_hat

```



```

def do_adam() :
    w_b_dw_db = [(init_w, init_b, 0, 0)]
    w_history, b_history, error_history = [], [], [
        ], []

    w, b, eta, mini_batch_size, num_points_seen =
        init_w, init_b, 0.1, 10, 0
    m_w, m_b, v_w, v_b, m_w_hat, m_b_hat, v_w_hat,
        v_b_hat, eps, beta1, beta2 = 0, 0, 0, 0, 0,
        0, 0, 0, 1e-8, 0.9, 0.999
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        m_w = beta1 * m_w + (1-beta1)*dw
        m_b = beta1 * m_b + (1-beta1)*db

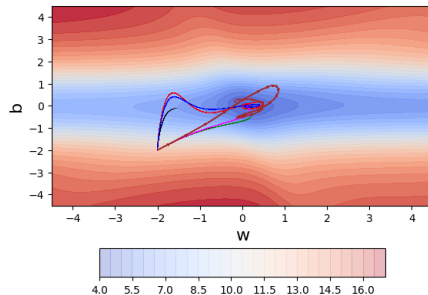
        v_w = beta2 * v_w + (1-beta2)*dw**2
        v_b = beta2 * v_b + (1-beta2)*db**2

        m_w_hat = m_w/(1-math.pow(beta1,i+1))
        m_b_hat = m_b/(1-math.pow(beta1,i+1))

        v_w_hat = v_w/(1-math.pow(beta2,i+1))
        v_b_hat = v_b/(1-math.pow(beta2,i+1))

        w = w - (eta / np.sqrt(v_w_hat + eps)) *
            m_w_hat
        b = b - (eta / np.sqrt(v_b_hat + eps)) *
            m_b_hat

```



```

def do_adam() :
    w_b_dw_db = [(init_w, init_b, 0, 0)]
    w_history, b_history, error_history = [], [], []

    w, b, eta, mini_batch_size, num_points_seen =
        init_w, init_b, 0.1, 10, 0
    m_w, m_b, v_w, v_b, m_w_hat, m_b_hat, v_w_hat,
        v_b_hat, eps, beta1, beta2 = 0, 0, 0, 0, 0,
        0, 0, 0, 1e-8, 0.9, 0.999
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        m_w = beta1 * m_w + (1-beta1)*dw
        m_b = beta1 * m_b + (1-beta1)*db

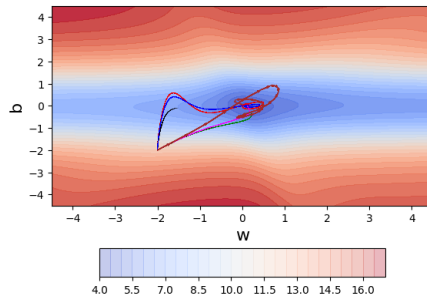
        v_w = beta2 * v_w + (1-beta2)*dw**2
        v_b = beta2 * v_b + (1-beta2)*db**2

        m_w_hat = m_w/(1-math.pow(beta1,i+1))
        m_b_hat = m_b/(1-math.pow(beta1,i+1))

        v_w_hat = v_w/(1-math.pow(beta2,i+1))
        v_b_hat = v_b/(1-math.pow(beta2,i+1))

        w = w - (eta / np.sqrt(v_w_hat + eps)) *
            m_w_hat
        b = b - (eta / np.sqrt(v_b_hat + eps)) *
            m_b_hat

```



```

def do_adam() :
    w_b_dw_db = [(init_w, init_b, 0, 0)]
    w_history, b_history, error_history = [], [], [
        ], []

    w, b, eta, mini_batch_size, num_points_seen =
        init_w, init_b, 0.1, 10, 0
    m_w, m_b, v_w, v_b, m_w_hat, m_b_hat, v_w_hat,
        v_b_hat, eps, beta1, beta2 = 0, 0, 0, 0, 0,
        0, 0, 0, 1e-8, 0.9, 0.999
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        m_w = beta1 * m_w + (1-beta1)*dw
        m_b = beta1 * m_b + (1-beta1)*db

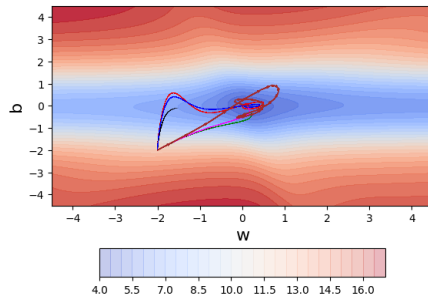
        v_w = beta2 * v_w + (1-beta2)*dw**2
        v_b = beta2 * v_b + (1-beta2)*db**2

        m_w_hat = m_w/(1-math.pow(beta1,i+1))
        m_b_hat = m_b/(1-math.pow(beta1,i+1))

        v_w_hat = v_w/(1-math.pow(beta2,i+1))
        v_b_hat = v_b/(1-math.pow(beta2,i+1))

        w = w - (eta / np.sqrt(v_w_hat + eps)) *
            m_w_hat
        b = b - (eta / np.sqrt(v_b_hat + eps)) *
            m_b_hat

```



```

def do_adam() :
    w_b_dw_db = [(init_w, init_b, 0, 0)]
    w_history, b_history, error_history = [], [], []

    w, b, eta, mini_batch_size, num_points_seen =
        init_w, init_b, 0.1, 10, 0
    m_w, m_b, v_w, v_b, m_w_hat, m_b_hat, v_w_hat,
        v_b_hat, eps, beta1, beta2 = 0, 0, 0, 0, 0,
        0, 0, 0, 1e-8, 0.9, 0.999
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        m_w = beta1 * m_w + (1-beta1)*dw
        m_b = beta1 * m_b + (1-beta1)*db

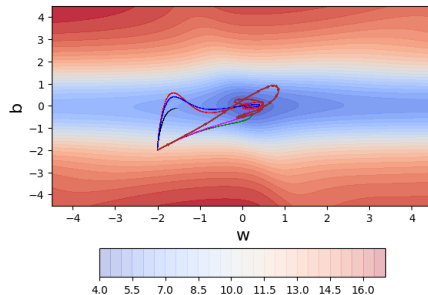
        v_w = beta2 * v_w + (1-beta2)*dw**2
        v_b = beta2 * v_b + (1-beta2)*db**2

        m_w_hat = m_w/(1-math.pow(beta1,i+1))
        m_b_hat = m_b/(1-math.pow(beta1,i+1))

        v_w_hat = v_w/(1-math.pow(beta2,i+1))
        v_b_hat = v_b/(1-math.pow(beta2,i+1))

        w = w - (eta / np.sqrt(v_w_hat + eps)) *
            m_w_hat
        b = b - (eta / np.sqrt(v_b_hat + eps)) *
            m_b_hat

```



```

def do_adam() :
    w_b_dw_db = [(init_w, init_b, 0, 0)]
    w_history, b_history, error_history = [], [], [
        ], []

    w, b, eta, mini_batch_size, num_points_seen =
        init_w, init_b, 0.1, 10, 0
    m_w, m_b, v_w, v_b, m_w_hat, m_b_hat, v_w_hat,
        v_b_hat, eps, beta1, beta2 = 0, 0, 0, 0, 0,
        0, 0, 0, 1e-8, 0.9, 0.999
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        m_w = beta1 * m_w + (1-beta1)*dw
        m_b = beta1 * m_b + (1-beta1)*db

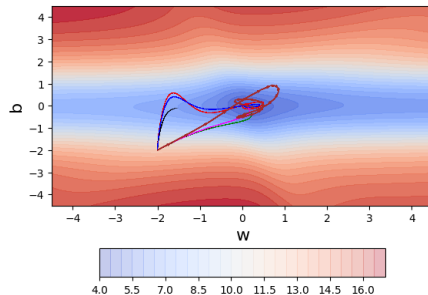
        v_w = beta2 * v_w + (1-beta2)*dw**2
        v_b = beta2 * v_b + (1-beta2)*db**2

        m_w_hat = m_w/(1-math.pow(beta1,i+1))
        m_b_hat = m_b/(1-math.pow(beta1,i+1))

        v_w_hat = v_w/(1-math.pow(beta2,i+1))
        v_b_hat = v_b/(1-math.pow(beta2,i+1))

        w = w - (eta / np.sqrt(v_w_hat + eps)) *
            m_w_hat
        b = b - (eta / np.sqrt(v_b_hat + eps)) *
            m_b_hat

```



```

def do_adam() :
    w_b_dw_db = [(init_w, init_b, 0, 0)]
    w_history, b_history, error_history = [], [], [
    ], []

    w, b, eta, mini_batch_size, num_points_seen =
        init_w, init_b, 0.1, 10, 0
    m_w, m_b, v_w, v_b, m_w_hat, m_b_hat, v_w_hat,
        v_b_hat, eps, beta1, beta2 = 0, 0, 0, 0, 0,
        0, 0, 0, 1e-8, 0.9, 0.999
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        m_w = beta1 * m_w + (1-beta1)*dw
        m_b = beta1 * m_b + (1-beta1)*db

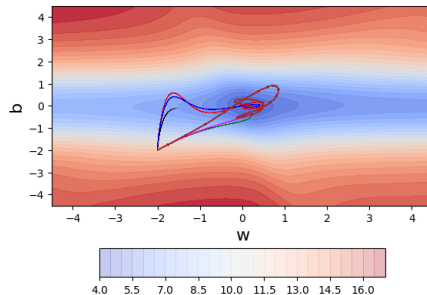
        v_w = beta2 * v_w + (1-beta2)*dw**2
        v_b = beta2 * v_b + (1-beta2)*db**2

        m_w_hat = m_w/(1-math.pow(beta1,i+1))
        m_b_hat = m_b/(1-math.pow(beta1,i+1))

        v_w_hat = v_w/(1-math.pow(beta2,i+1))
        v_b_hat = v_b/(1-math.pow(beta2,i+1))

        w = w - (eta / np.sqrt(v_w_hat + eps)) *
            m_w_hat
        b = b - (eta / np.sqrt(v_b_hat + eps)) *
            m_b_hat

```




```

def do_adam() :
    w_b_dw_db = [(init_w, init_b, 0, 0)]
    w_history, b_history, error_history = [], [], [
        ], []

    w, b, eta, mini_batch_size, num_points_seen =
        init_w, init_b, 0.1, 10, 0
    m_w, m_b, v_w, v_b, m_w_hat, m_b_hat, v_w_hat,
        v_b_hat, eps, beta1, beta2 = 0, 0, 0, 0, 0,
        0, 0, 0, 1e-8, 0.9, 0.999
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        m_w = beta1 * m_w + (1-beta1)*dw
        m_b = beta1 * m_b + (1-beta1)*db

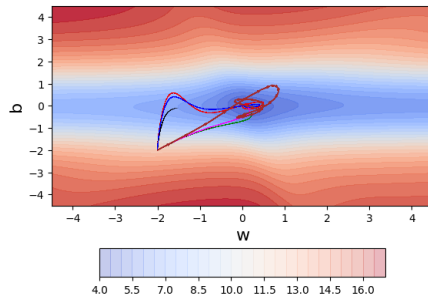
        v_w = beta2 * v_w + (1-beta2)*dw**2
        v_b = beta2 * v_b + (1-beta2)*db**2

        m_w_hat = m_w/(1-math.pow(beta1,i+1))
        m_b_hat = m_b/(1-math.pow(beta1,i+1))

        v_w_hat = v_w/(1-math.pow(beta2,i+1))
        v_b_hat = v_b/(1-math.pow(beta2,i+1))

        w = w - (eta / np.sqrt(v_w_hat + eps)) *
            m_w_hat
        b = b - (eta / np.sqrt(v_b_hat + eps)) *
            m_b_hat

```



```

def do_adam() :
    w_b_dw_db = [(init_w, init_b, 0, 0)]
    w_history, b_history, error_history = [], [], [
        ], []

    w, b, eta, mini_batch_size, num_points_seen =
        init_w, init_b, 0.1, 10, 0
    m_w, m_b, v_w, v_b, m_w_hat, m_b_hat, v_w_hat,
        v_b_hat, eps, beta1, beta2 = 0, 0, 0, 0, 0,
        0, 0, 0, 1e-8, 0.9, 0.999
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        m_w = beta1 * m_w + (1-beta1)*dw
        m_b = beta1 * m_b + (1-beta1)*db

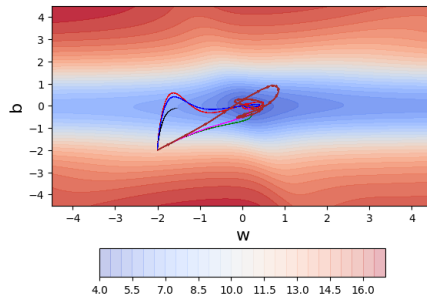
        v_w = beta2 * v_w + (1-beta2)*dw**2
        v_b = beta2 * v_b + (1-beta2)*db**2

        m_w_hat = m_w/(1-math.pow(beta1,i+1))
        m_b_hat = m_b/(1-math.pow(beta1,i+1))

        v_w_hat = v_w/(1-math.pow(beta2,i+1))
        v_b_hat = v_b/(1-math.pow(beta2,i+1))

        w = w - (eta / np.sqrt(v_w_hat + eps)) *
            m_w_hat
        b = b - (eta / np.sqrt(v_b_hat + eps)) *
            m_b_hat

```



```

def do_adam() :
    w_b_dw_db = [(init_w, init_b, 0, 0)]
    w_history, b_history, error_history = [], [], [
    ], []

    w, b, eta, mini_batch_size, num_points_seen =
        init_w, init_b, 0.1, 10, 0
    m_w, m_b, v_w, v_b, m_w_hat, m_b_hat, v_w_hat,
        v_b_hat, eps, beta1, beta2 = 0, 0, 0, 0, 0,
        0, 0, 0, 1e-8, 0.9, 0.999
    for i in range(max_epochs) :
        dw, db = 0, 0
        for x,y in zip(X, Y) :
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)

        m_w = beta1 * m_w + (1-beta1)*dw
        m_b = beta1 * m_b + (1-beta1)*db

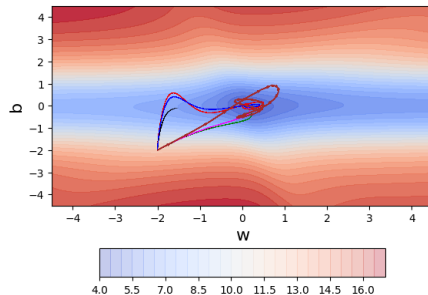
        v_w = beta2 * v_w + (1-beta2)*dw**2
        v_b = beta2 * v_b + (1-beta2)*db**2

        m_w_hat = m_w/(1-math.pow(beta1,i+1))
        m_b_hat = m_b/(1-math.pow(beta1,i+1))

        v_w_hat = v_w/(1-math.pow(beta2,i+1))
        v_b_hat = v_b/(1-math.pow(beta2,i+1))

        w = w - (eta / np.sqrt(v_w_hat + eps)) *
            m_w_hat
        b = b - (eta / np.sqrt(v_b_hat + eps)) *
            m_b_hat

```



- As expected, taking a cumulative history gives a speed up ...

Million dollar question: Which algorithm to use in practice

- Adam seems to be more or less the default choice now ($\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 1e-8$)

Million dollar question: Which algorithm to use in practice

- Adam seems to be more or less the default choice now ($\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 1e - 8$)
- Although it is supposed to be robust to initial learning rates, we have observed that for sequence generation problems $\eta = 0.001, 0.0001$ works best

Million dollar question: Which algorithm to use in practice

- Adam seems to be more or less the default choice now ($\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 1e - 8$)
- Although it is supposed to be robust to initial learning rates, we have observed that for sequence generation problems $\eta = 0.001, 0.0001$ works best
- Having said that, many papers report that SGD with momentum (Nesterov or classical) with a simple annealing learning rate schedule also works well in practice (typically, starting with $\eta = 0.001, 0.0001$ for sequence generation problems)

Million dollar question: Which algorithm to use in practice

- Adam seems to be more or less the default choice now ($\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 1e - 8$)
- Although it is supposed to be robust to initial learning rates, we have observed that for sequence generation problems $\eta = 0.001, 0.0001$ works best
- Having said that, many papers report that SGD with momentum (Nesterov or classical) with a simple annealing learning rate schedule also works well in practice (typically, starting with $\eta = 0.001, 0.0001$ for sequence generation problems)
- Adam might just be the best choice overall!!

Million dollar question: Which algorithm to use in practice

- Adam seems to be more or less the default choice now ($\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 1e - 8$)
- Although it is supposed to be robust to initial learning rates, we have observed that for sequence generation problems $\eta = 0.001, 0.0001$ works best
- Having said that, many papers report that SGD with momentum (Nesterov or classical) with a simple annealing learning rate schedule also works well in practice (typically, starting with $\eta = 0.001, 0.0001$ for sequence generation problems)
- Adam might just be the best choice overall!!
- Some recent work suggest that there is a problem with Adam and it will not converge in some cases

Explanation for why we need bias correction in Adam

Update rule for Adam

$$m_t = \beta_1 * m_{t-1} + (1 - \beta_1) * \nabla w_t$$

$$v_t = \beta_2 * v_{t-1} + (1 - \beta_2) * (\nabla w_t)^2$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} * \hat{m}_t$$

- Note that we are taking a running average of the gradients as m_t

Update rule for Adam

$$m_t = \beta_1 * m_{t-1} + (1 - \beta_1) * \nabla w_t$$

$$v_t = \beta_2 * v_{t-1} + (1 - \beta_2) * (\nabla w_t)^2$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} * \hat{m}_t$$

- Note that we are taking a running average of the gradients as m_t
- The reason we are doing this is that we don't want to rely too much on the current gradient and instead rely on the overall behaviour of the gradients over many timesteps

Update rule for Adam

$$m_t = \beta_1 * m_{t-1} + (1 - \beta_1) * \nabla w_t$$

$$v_t = \beta_2 * v_{t-1} + (1 - \beta_2) * (\nabla w_t)^2$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} * \hat{m}_t$$

- Note that we are taking a running average of the gradients as m_t
- The reason we are doing this is that we don't want to rely too much on the current gradient and instead rely on the overall behaviour of the gradients over many timesteps
- One way of looking at this is that we are interested in the expected value of the gradients and not on a single point estimate computed at time t

Update rule for Adam

$$m_t = \beta_1 * m_{t-1} + (1 - \beta_1) * \nabla w_t$$

$$v_t = \beta_2 * v_{t-1} + (1 - \beta_2) * (\nabla w_t)^2$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} * \hat{m}_t$$

- Note that we are taking a running average of the gradients as m_t
- The reason we are doing this is that we don't want to rely too much on the current gradient and instead rely on the overall behaviour of the gradients over many timesteps
- One way of looking at this is that we are interested in the expected value of the gradients and not on a single point estimate computed at time t
- However, instead of computing $E[\nabla w_t]$ we are computing m_t as the exponentially moving average

Update rule for Adam

$$m_t = \beta_1 * m_{t-1} + (1 - \beta_1) * \nabla w_t$$

$$v_t = \beta_2 * v_{t-1} + (1 - \beta_2) * (\nabla w_t)^2$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} * \hat{m}_t$$

- Note that we are taking a running average of the gradients as m_t
- The reason we are doing this is that we don't want to rely too much on the current gradient and instead rely on the overall behaviour of the gradients over many timesteps
- One way of looking at this is that we are interested in the expected value of the gradients and not on a single point estimate computed at time t
- However, instead of computing $E[\nabla w_t]$ we are computing m_t as the exponentially moving average
- Ideally we would want $E[m_t]$ to be equal to $E[\nabla w_t]$

Update rule for Adam

$$m_t = \beta_1 * m_{t-1} + (1 - \beta_1) * \nabla w_t$$

$$v_t = \beta_2 * v_{t-1} + (1 - \beta_2) * (\nabla w_t)^2$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t + \epsilon}} * \hat{m}_t$$

- Note that we are taking a running average of the gradients as m_t
- The reason we are doing this is that we don't want to rely too much on the current gradient and instead rely on the overall behaviour of the gradients over many timesteps
- One way of looking at this is that we are interested in the expected value of the gradients and not on a single point estimate computed at time t
- However, instead of computing $E[\nabla w_t]$ we are computing m_t as the exponentially moving average
- Ideally we would want $E[m_t]$ to be equal to $E[\nabla w_t]$
- Let us see if that is the case

- For convenience we will denote ∇w_t as g_t and β_1 as β

- For convenience we will denote ∇w_t as g_t and β_1 as β

$$m_t = \beta * m_{t-1} + (1 - \beta) * g_t$$

- For convenience we will denote ∇w_t as g_t and β_1 as β

$$m_t = \beta * m_{t-1} + (1 - \beta) * g_t$$

$$m_0 = 0$$

- For convenience we will denote ∇w_t as g_t and β_1 as β

$$m_t = \beta * m_{t-1} + (1 - \beta) * g_t$$

$$m_0 = 0$$

$$m_1 = \beta m_0 + (1 - \beta) g_1$$

- For convenience we will denote ∇w_t as g_t and β_1 as β

$$m_t = \beta * m_{t-1} + (1 - \beta) * g_t$$

$$m_0 = 0$$

$$\begin{aligned} m_1 &= \beta m_0 + (1 - \beta) g_1 \\ &= (1 - \beta) g_1 \end{aligned}$$

- For convenience we will denote ∇w_t as g_t and β_1 as β

$$m_t = \beta * m_{t-1} + (1 - \beta) * g_t$$

$$m_0 = 0$$

$$m_1 = \beta m_0 + (1 - \beta) g_1$$

$$= (1 - \beta) g_1$$

$$m_2 = \beta m_1 + (1 - \beta) g_2$$

- For convenience we will denote ∇w_t as g_t and β_1 as β

$$m_t = \beta * m_{t-1} + (1 - \beta) * g_t$$

$$m_0 = 0$$

$$m_1 = \beta m_0 + (1 - \beta) g_1$$

$$= (1 - \beta) g_1$$

$$m_2 = \beta m_1 + (1 - \beta) g_2$$

$$= \beta(1 - \beta) g_1 + (1 - \beta) g_2$$

- For convenience we will denote ∇w_t as g_t and β_1 as β

$$m_t = \beta * m_{t-1} + (1 - \beta) * g_t$$

$$m_0 = 0$$

$$m_1 = \beta m_0 + (1 - \beta) g_1$$

$$= (1 - \beta) g_1$$

$$m_2 = \beta m_1 + (1 - \beta) g_2$$

$$= \beta(1 - \beta) g_1 + (1 - \beta) g_2$$

$$m_3 = \beta m_2 + (1 - \beta) g_3$$

- For convenience we will denote ∇w_t as g_t and β_1 as β

$$m_t = \beta * m_{t-1} + (1 - \beta) * g_t$$

$$m_0 = 0$$

$$m_1 = \beta m_0 + (1 - \beta) g_1$$

$$= (1 - \beta) g_1$$

$$m_2 = \beta m_1 + (1 - \beta) g_2$$

$$= \beta(1 - \beta) g_1 + (1 - \beta) g_2$$

$$m_3 = \beta m_2 + (1 - \beta) g_3$$

$$= \beta(\beta(1 - \beta) g_1 + (1 - \beta) g_2) + (1 - \beta) g_3$$

- For convenience we will denote ∇w_t as g_t and β_1 as β

$$m_t = \beta * m_{t-1} + (1 - \beta) * g_t$$

$$m_0 = 0$$

$$m_1 = \beta m_0 + (1 - \beta) g_1$$

$$= (1 - \beta) g_1$$

$$m_2 = \beta m_1 + (1 - \beta) g_2$$

$$= \beta(1 - \beta) g_1 + (1 - \beta) g_2$$

$$m_3 = \beta m_2 + (1 - \beta) g_3$$

$$= \beta(\beta(1 - \beta) g_1 + (1 - \beta) g_2) + (1 - \beta) g_3$$

$$= \beta^2(1 - \beta) g_1 + \beta(1 - \beta) g_2 + (1 - \beta) g_3$$

- For convenience we will denote ∇w_t as g_t and β_1 as β

$$m_t = \beta * m_{t-1} + (1 - \beta) * g_t$$

$$m_0 = 0$$

$$m_1 = \beta m_0 + (1 - \beta) g_1$$

$$= (1 - \beta) g_1$$

$$m_2 = \beta m_1 + (1 - \beta) g_2$$

$$= \beta(1 - \beta) g_1 + (1 - \beta) g_2$$

$$m_3 = \beta m_2 + (1 - \beta) g_3$$

$$= \beta(\beta(1 - \beta) g_1 + (1 - \beta) g_2) + (1 - \beta) g_3$$

$$= \beta^2(1 - \beta) g_1 + \beta(1 - \beta) g_2 + (1 - \beta) g_3$$

$$= (1 - \beta) \sum_{i=1}^3 \beta^{3-i} g_i$$

- For convenience we will denote ∇w_t as g_t and β_1 as β

$$m_t = \beta * m_{t-1} + (1 - \beta) * g_t$$

$$m_0 = 0$$

$$m_1 = \beta m_0 + (1 - \beta) g_1$$

$$= (1 - \beta) g_1$$

$$m_2 = \beta m_1 + (1 - \beta) g_2$$

$$= \beta(1 - \beta) g_1 + (1 - \beta) g_2$$

$$m_3 = \beta m_2 + (1 - \beta) g_3$$

$$= \beta(\beta(1 - \beta) g_1 + (1 - \beta) g_2) + (1 - \beta) g_3$$

$$= \beta^2(1 - \beta) g_1 + \beta(1 - \beta) g_2 + (1 - \beta) g_3$$

$$= (1 - \beta) \sum_{i=1}^3 \beta^{3-i} g_i$$

- In general,

$$m_t = (1 - \beta) \sum_{i=1}^t \beta^{t-i} g_i$$

- So we have, $m_t = (1 - \beta) \sum_{i=1}^t \beta^{t-i} g_i$

- So we have, $m_t = (1 - \beta) \sum_{i=1}^t \beta^{t-i} g_i$
- Taking Expectation on both sides

- So we have, $m_t = (1 - \beta) \sum_{i=1}^t \beta^{t-i} g_i$
- Taking Expectation on both sides

$$E[m_t] = E[(1 - \beta) \sum_{i=1}^t \beta^{t-i} g_i]$$

- So we have, $m_t = (1 - \beta) \sum_{i=1}^t \beta^{t-i} g_i$
- Taking Expectation on both sides

$$E[m_t] = E[(1 - \beta) \sum_{i=1}^t \beta^{t-i} g_i]$$

$$E[m_t] = (1 - \beta) E[\sum_{i=1}^t \beta^{t-i} g_i]$$

- So we have, $m_t = (1 - \beta) \sum_{i=1}^t \beta^{t-i} g_i$
- Taking Expectation on both sides

$$E[m_t] = E[(1 - \beta) \sum_{i=1}^t \beta^{t-i} g_i]$$

$$E[m_t] = (1 - \beta) E[\sum_{i=1}^t \beta^{t-i} g_i]$$

$$E[m_t] = (1 - \beta) \sum_{i=1}^t E[\beta^{t-i} g_i]$$

- So we have, $m_t = (1 - \beta) \sum_{i=1}^t \beta^{t-i} g_i$
- Taking Expectation on both sides

$$E[m_t] = E[(1 - \beta) \sum_{i=1}^t \beta^{t-i} g_i]$$

$$E[m_t] = (1 - \beta) E[\sum_{i=1}^t \beta^{t-i} g_i]$$

$$\begin{aligned} E[m_t] &= (1 - \beta) \sum_{i=1}^t E[\beta^{t-i} g_i] \\ &= (1 - \beta) \sum_{i=1}^t \beta^{t-i} E[g_i] \end{aligned}$$

- So we have, $m_t = (1 - \beta) \sum_{i=1}^t \beta^{t-i} g_i$
- Taking Expectation on both sides

$$E[m_t] = E[(1 - \beta) \sum_{i=1}^t \beta^{t-i} g_i]$$

$$E[m_t] = (1 - \beta) E[\sum_{i=1}^t \beta^{t-i} g_i]$$

$$\begin{aligned} E[m_t] &= (1 - \beta) \sum_{i=1}^t E[\beta^{t-i} g_i] \\ &= (1 - \beta) \sum_{i=1}^t \beta^{t-i} E[g_i] \end{aligned}$$

- Assumption: All g_i 's come from the same distribution i.e. $E[g_i] = E[g] \forall i$

- So we have, $m_t = (1 - \beta) \sum_{i=1}^t \beta^{t-i} g_i$
- Taking Expectation on both sides

$$E[m_t] = E[(1 - \beta) \sum_{i=1}^t \beta^{t-i} g_i]$$

$$E[m_t] = (1 - \beta) E[\sum_{i=1}^t \beta^{t-i} g_i]$$

$$\begin{aligned} E[m_t] &= (1 - \beta) \sum_{i=1}^t E[\beta^{t-i} g_i] \\ &= (1 - \beta) \sum_{i=1}^t \beta^{t-i} E[g_i] \end{aligned}$$

- Assumption: All g_i 's come from the same distribution i.e. $E[g_i] = E[g] \forall i$

$$E[m_t] = (1 - \beta) \sum_{i=1}^t (\beta)^{t-i} E[g_i]$$

- So we have, $m_t = (1 - \beta) \sum_{i=1}^t \beta^{t-i} g_i$
- Taking Expectation on both sides

$$E[m_t] = E[(1 - \beta) \sum_{i=1}^t \beta^{t-i} g_i]$$

$$E[m_t] = (1 - \beta) E[\sum_{i=1}^t \beta^{t-i} g_i]$$

$$\begin{aligned} E[m_t] &= (1 - \beta) \sum_{i=1}^t E[\beta^{t-i} g_i] \\ &= (1 - \beta) \sum_{i=1}^t \beta^{t-i} E[g_i] \end{aligned}$$

- Assumption: All g_i 's come from the same distribution i.e. $E[g_i] = E[g] \forall i$

$$\begin{aligned} E[m_t] &= (1 - \beta) \sum_{i=1}^t (\beta)^{t-i} E[g_i] \\ &= E[g] (1 - \beta) \sum_{i=1}^t (\beta)^{t-i} \end{aligned}$$

- So we have, $m_t = (1 - \beta) \sum_{i=1}^t \beta^{t-i} g_i$
- Taking Expectation on both sides

$$E[m_t] = E[(1 - \beta) \sum_{i=1}^t \beta^{t-i} g_i]$$

$$E[m_t] = (1 - \beta) E[\sum_{i=1}^t \beta^{t-i} g_i]$$

$$\begin{aligned} E[m_t] &= (1 - \beta) \sum_{i=1}^t E[\beta^{t-i} g_i] \\ &= (1 - \beta) \sum_{i=1}^t \beta^{t-i} E[g_i] \end{aligned}$$

- Assumption: All g_i 's come from the same distribution i.e. $E[g_i] = E[g] \forall i$

$$E[m_t] = (1 - \beta) \sum_{i=1}^t (\beta)^{t-i} E[g_i]$$

$$= E[g] (1 - \beta) \sum_{i=1}^t (\beta)^{t-i}$$

$$= E[g] (1 - \beta) (\beta^{t-1} + \beta^{t-2} + \dots + \beta^0)$$

- So we have, $m_t = (1 - \beta) \sum_{i=1}^t \beta^{t-i} g_i$
- Taking Expectation on both sides

$$E[m_t] = E[(1 - \beta) \sum_{i=1}^t \beta^{t-i} g_i]$$

$$E[m_t] = (1 - \beta) E[\sum_{i=1}^t \beta^{t-i} g_i]$$

$$\begin{aligned} E[m_t] &= (1 - \beta) \sum_{i=1}^t E[\beta^{t-i} g_i] \\ &= (1 - \beta) \sum_{i=1}^t \beta^{t-i} E[g_i] \end{aligned}$$

- Assumption: All g_i 's come from the same distribution i.e. $E[g_i] = E[g] \forall i$

$$E[m_t] = (1 - \beta) \sum_{i=1}^t (\beta)^{t-i} E[g_i]$$

$$\begin{aligned} &= E[g] (1 - \beta) \sum_{i=1}^t (\beta)^{t-i} \\ &= E[g] (1 - \beta) (\beta^{t-1} + \beta^{t-2} + \dots + \beta^0) \\ &= E[g] (1 - \beta) \frac{1 - \beta^t}{1 - \beta} \end{aligned}$$

- So we have, $m_t = (1 - \beta) \sum_{i=1}^t \beta^{t-i} g_i$
- Taking Expectation on both sides

$$E[m_t] = E[(1 - \beta) \sum_{i=1}^t \beta^{t-i} g_i]$$

$$E[m_t] = (1 - \beta) E[\sum_{i=1}^t \beta^{t-i} g_i]$$

$$\begin{aligned} E[m_t] &= (1 - \beta) \sum_{i=1}^t E[\beta^{t-i} g_i] \\ &= (1 - \beta) \sum_{i=1}^t \beta^{t-i} E[g_i] \end{aligned}$$

- Assumption: All g_i 's come from the same distribution i.e. $E[g_i] = E[g] \forall i$

$$E[m_t] = (1 - \beta) \sum_{i=1}^t (\beta)^{t-i} E[g_i]$$

$$\begin{aligned} &= E[g] (1 - \beta) \sum_{i=1}^t (\beta)^{t-i} \\ &= E[g] (1 - \beta) (\beta^{t-1} + \beta^{t-2} + \dots + \beta^0) \\ &= E[g] (1 - \beta) \frac{1 - \beta^t}{1 - \beta} \end{aligned}$$

the last fraction is the sum of a GP with common ratio = β

- So we have, $m_t = (1 - \beta) \sum_{i=1}^t \beta^{t-i} g_i$
- Taking Expectation on both sides

$$E[m_t] = E[(1 - \beta) \sum_{i=1}^t \beta^{t-i} g_i]$$

$$E[m_t] = (1 - \beta) E[\sum_{i=1}^t \beta^{t-i} g_i]$$

$$\begin{aligned} E[m_t] &= (1 - \beta) \sum_{i=1}^t E[\beta^{t-i} g_i] \\ &= (1 - \beta) \sum_{i=1}^t \beta^{t-i} E[g_i] \end{aligned}$$

- Assumption: All g_i 's come from the same distribution i.e. $E[g_i] = E[g] \forall i$

$$E[m_t] = (1 - \beta) \sum_{i=1}^t (\beta)^{t-i} E[g_i]$$

$$\begin{aligned} &= E[g] (1 - \beta) \sum_{i=1}^t (\beta)^{t-i} \\ &= E[g] (1 - \beta) (\beta^{t-1} + \beta^{t-2} + \dots + \beta^0) \\ &= E[g] (1 - \beta) \frac{1 - \beta^t}{1 - \beta} \end{aligned}$$

the last fraction is the sum of a GP with common ratio = β

$$E[m_t] = E[g] (1 - \beta^t)$$

- So we have, $m_t = (1 - \beta) \sum_{i=1}^t \beta^{t-i} g_i$
- Taking Expectation on both sides

$$E[m_t] = E[(1 - \beta) \sum_{i=1}^t \beta^{t-i} g_i]$$

$$E[m_t] = (1 - \beta) E[\sum_{i=1}^t \beta^{t-i} g_i]$$

$$\begin{aligned} E[m_t] &= (1 - \beta) \sum_{i=1}^t E[\beta^{t-i} g_i] \\ &= (1 - \beta) \sum_{i=1}^t \beta^{t-i} E[g_i] \end{aligned}$$

- Assumption: All g_i 's come from the same distribution i.e. $E[g_i] = E[g] \forall i$

$$E[m_t] = (1 - \beta) \sum_{i=1}^t (\beta)^{t-i} E[g_i]$$

$$\begin{aligned} &= E[g] (1 - \beta) \sum_{i=1}^t (\beta)^{t-i} \\ &= E[g] (1 - \beta) (\beta^{t-1} + \beta^{t-2} + \dots + \beta^0) \\ &= E[g] (1 - \beta) \frac{1 - \beta^t}{1 - \beta} \end{aligned}$$

the last fraction is the sum of a GP with common ratio = β

$$\begin{aligned} E[m_t] &= E[g] (1 - \beta^t) \\ E\left[\frac{m_t}{1 - \beta^t}\right] &= E[g] \end{aligned}$$

- So we have, $m_t = (1 - \beta) \sum_{i=1}^t \beta^{t-i} g_i$
- Taking Expectation on both sides

$$E[m_t] = E[(1 - \beta) \sum_{i=1}^t \beta^{t-i} g_i]$$

$$E[m_t] = (1 - \beta) E[\sum_{i=1}^t \beta^{t-i} g_i]$$

$$\begin{aligned} E[m_t] &= (1 - \beta) \sum_{i=1}^t E[\beta^{t-i} g_i] \\ &= (1 - \beta) \sum_{i=1}^t \beta^{t-i} E[g_i] \end{aligned}$$

- Assumption: All g_i 's come from the same distribution i.e. $E[g_i] = E[g] \forall i$

$$E[m_t] = (1 - \beta) \sum_{i=1}^t (\beta)^{t-i} E[g_i]$$

$$\begin{aligned} &= E[g] (1 - \beta) \sum_{i=1}^t (\beta)^{t-i} \\ &= E[g] (1 - \beta) (\beta^{t-1} + \beta^{t-2} + \dots + \beta^0) \\ &= E[g] (1 - \beta) \frac{1 - \beta^t}{1 - \beta} \end{aligned}$$

the last fraction is the sum of a GP with common ratio = β

$$\begin{aligned} E[m_t] &= E[g] (1 - \beta^t) \\ E\left[\frac{m_t}{1 - \beta^t}\right] &= E[g] \\ E[\hat{m}_t] &= E[g] \left(\because \frac{m_t}{1 - \beta^t} = \hat{m}_t\right) \end{aligned}$$

- So we have, $m_t = (1 - \beta) \sum_{i=1}^t \beta^{t-i} g_i$
- Taking Expectation on both sides

$$E[m_t] = E[(1 - \beta) \sum_{i=1}^t \beta^{t-i} g_i]$$

$$E[m_t] = (1 - \beta) E[\sum_{i=1}^t \beta^{t-i} g_i]$$

$$\begin{aligned} E[m_t] &= (1 - \beta) \sum_{i=1}^t E[\beta^{t-i} g_i] \\ &= (1 - \beta) \sum_{i=1}^t \beta^{t-i} E[g_i] \end{aligned}$$

- Assumption: All g_i 's come from the same distribution i.e. $E[g_i] = E[g] \forall i$

$$E[m_t] = (1 - \beta) \sum_{i=1}^t (\beta)^{t-i} E[g_i]$$

$$\begin{aligned} &= E[g] (1 - \beta) \sum_{i=1}^t (\beta)^{t-i} \\ &= E[g] (1 - \beta) (\beta^{t-1} + \beta^{t-2} + \dots + \beta^0) \\ &= E[g] (1 - \beta) \frac{1 - \beta^t}{1 - \beta} \end{aligned}$$

the last fraction is the sum of a GP with common ratio = β

$$\begin{aligned} E[m_t] &= E[g] (1 - \beta^t) \\ E\left[\frac{m_t}{1 - \beta^t}\right] &= E[g] \\ E[\hat{m}_t] &= E[g] (\because \frac{m_t}{1 - \beta^t} = \hat{m}_t) \end{aligned}$$

Hence we apply the bias correction because then the expected value of $\hat{m}_t$ is the same as the expected value of g_t